Deep Learning: Making It Learnable

A course companion in native Python and PyTorch

Heman Shakeri

2026-07-08

Table of contents

Preface

> ⚠ Warning
>
> **Draft.** This book is being written in the open as the companion text for DS 6050 Deep Learning at UVA's School of Data Science. Chapters appear as they mature; everything here runs — every figure and result is produced by the code on the page.

Deep learning did not appear out of nowhere. Nearly every idea that powers modern AI is a classical idea — a line of best fit, a hand-designed image filter, a kernel-weighted average — that someone dared to ask one question about:

> *What if we made this* ***learnable****?*

That question is the spine of this book. We will ask it over and over:

- Take **linear regression**, let its features be learned, and you get the multilayer perceptron.
- Watch the MLP **fail to generalize** on images — see the failure in pictures — and the cure (respecting the structure of the data) becomes obvious: an **inductive bias**.
- Take the **fixed filters** of classical computer vision, make them learnable, and you get the convolutional network.
- Take **kernel regression** — a century-old way to average nearby evidence — make the similarity function learnable, and you get **attention**, then self-attention, then the Transformer.
- Then comes the **BERT moment**: stop training from scratch at all, and adapt what has already been learned.

Each part of the book replays this same move at a larger scale. By the end, the modern stack should feel less like a zoo of architectures and more like one idea, compounding.

How to read this book

Every chapter follows the course's learning loop: **read → run → check**. The code is native Python and PyTorch, written to run on an ordinary laptop CPU — small data, honest experiments. Watch the lecture videos, read the chapter, run the cells, and test yourself against the self-checks on the course site.

Acknowledgments

To the students of DS 6050, whose questions shaped every explanation here.

Part I.

I · From Lines to Networks

1. Linear Regression, the Mother Model

The core task of everything we will do in this course is to teach a computer to learn a function from examples. Before we can appreciate what is *deep* about deep learning, we need one complete, honest example of learning — a model, a loss, and a way to improve. Linear regression is that example. It is the smallest model that contains the entire supervised learning story, and by the end of this chapter we will have built it three times: once in closed form, once by gradient descent, and once the way you will build everything else in this book — with PyTorch modules. All three will agree, and you will know exactly why.

1.1. Learning from examples

We start with a dataset of examples,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n},$$

where each $\mathbf{x}_i \in \mathbb{R}^d$ is an input with d features and y_i is the desired output — the *supervision signal*. Stack the inputs as rows and you get the data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$: n samples down, d features across, with the targets collected in a vector $\mathbf{y}$.

Our goal is a flexible function f such that

$$f(\mathbf{x}_i) \approx y_i \quad \text{and, crucially, } f \text{ generalizes to unseen data.}$$

That second clause is the whole game. We do not care how well f recites the training examples; we care how it behaves on a new $\mathbf{x}$ it has never seen — one drawn from the same distribution as the training data.

Supervised learning, three flavors

The range of y decides the task and, as we will see, the natural loss:

Task	Range of y	Typical loss
Regression	$\mathbb{R}$	Mean squared error
Classification	$\{0, \dots, C-1\}$	Cross-entropy
Structured prediction	sequences, images, graphs	task-specific

This chapter is regression; Chapter 2 handles classification with the same machinery.

1.1.1. Why is this hard?

This task sounds simple, but it is fundamentally hard: for any finite set of examples, there are *infinitely many* functions that fit them perfectly. We want the one that is most suited to our problem — so the learning algorithm needs a nudge in the right direction. That guiding assumption is called its **inductive bias**, and it is a thread we will pull on for the rest of the book.

- A **linear model** has a *strong* bias: it assumes the world is linear. Simple, stable — and systematically wrong when the data is not linear (high bias, low variance).
- A **deep neural network** has a *weak* bias. Its flexibility lets it learn complex patterns, but it can memorize the training data instead of the underlying rule (low bias, high variance). Some networks can memorize an entire training set — and then perform poorly the day you deploy them.

The key to modern deep learning is to use a flexible model and fight overfitting with data, regularization, and — the deepest idea of all — inductive biases matched to the task. That last idea gets its own chapter (Chapter 6).

To make any of this concrete, we parameterize a family of functions $f_{\mathbf{w}}$ by a set of tuning parameters $\mathbf{w}$ — think of each parameter as a knob. *Learning* is finding a good setting $\hat{\mathbf{w}}$ of the knobs; *inference* is using $f_{\hat{\mathbf{w}}}$ to predict on unseen data. That is the vocabulary we will use all course.

1.2. The linear model and its geometry

Linear regression assumes a linear relationship:

$$y = \mathbf{w}^\top \mathbf{x} + b.$$

Geometrically, $\mathbf{w}$ sets the orientation — the slope — of a hyperplane, and the bias b shifts it away from the origin. In two dimensions this is the familiar line through a scatter of points; in d dimensions it is a d-dimensional plane, but nothing about the algebra changes.

1.2.1. The augmentation trick

Carrying b separately clutters the algebra, so we will often fold it into the weights: append a column of ones to the data matrix, making $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$, and append b to the weight vector, making $\mathbf{w} \in \mathbb{R}^{d+1}$. Then

$$y = \mathbf{w}^\top \mathbf{x} \qquad \text{(bias included)}.$$

Whenever you see a linear model written without a bias — here or inside a neural network — the bias has not disappeared; it is living inside $\mathbf{w}$.

1.3. The loss: what makes a hyperplane "bad"?

To find the best hyperplane we must first quantify error. The standard choice for regression is the **mean squared error (MSE)**:

$$\mathcal{L}(\mathbf{w}) \;=\; \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2 \;=\; \frac{1}{n}\,\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2\,. \tag{1.1}$$

The loss is the guiding principle of training: it is how we tell the model *you are a bad model, and here is by how much* — and the model has to find a way to improve. The choice of loss is not arbitrary, and we will justify this one honestly at the end of the chapter.

> The first "make it learnable" seed
>
> Look closely at $y = \mathbf{w}^\top \mathbf{x} + b$. A single neuron in a neural network computes *exactly this*, followed by a nonlinear squashing function $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Linear regression **is** a neuron with the identity activation. Everything we learn here — loss, gradients, optimization — transfers directly. The rest of this book is, in a precise sense, the story of what happens when we take this fixed, simple recipe and progressively let more of it be *learned*.

1.4. Finding the best weights, method 1: solve it exactly

For this one special problem we can find the exact minimizer. Set the gradient of Equation 1.1 to zero and you get the **normal equations**:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}. \tag{1.2}$$

There is a picture behind this formula worth carrying for the rest of your career. Our predictions $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ can only ever live in the **column space** of $\mathbf{X}$ — the span of its feature columns. If the true $\mathbf{y}$ lies outside that space (it almost always does), the best we can do is **project** $\mathbf{y}$ onto it. The leftover error $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is what our model cannot explain, and at the optimum it is *perpendicular* to the column space — no adjustment of the knobs can reduce it further.

Elegant — so why is this formula almost absent from deep learning? Two reasons. Solving it costs $O(d^3)$, which is hopeless when d runs to millions or billions. And it only exists because the model is linear; the moment we add a nonlinearity, there is no closed form to solve. Our datasets are large and our models are not linear — we have neither luxury. We need another way.

1.5. Finding the best weights, method 2: walk downhill

Here is the geometric intuition. Picture the loss as a landscape over the parameter space, and imagine standing on it *blindfolded*, trying to find the lowest valley. You cannot see the valley —

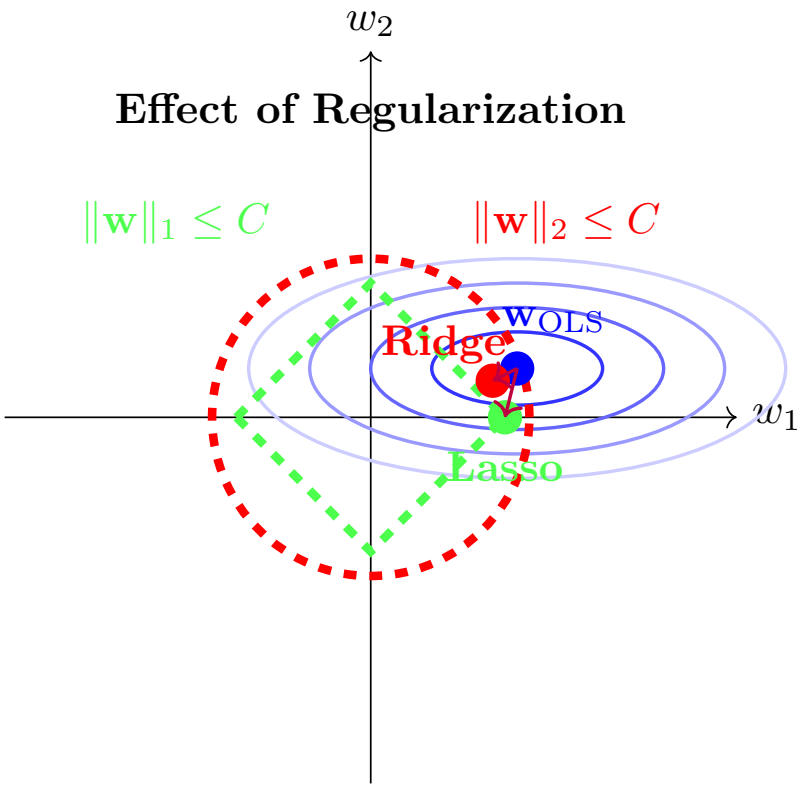

Figure 1.1.: The normal equations, geometrically: the optimal prediction $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$ is the projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual $\mathbf{e}$ is orthogonal to it.

but you can feel the slope under your feet. So you feel the slope, take a small step downhill, and repeat.

The mathematical form of this hill-descending intuition is **gradient descent**:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \, \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t)}), \tag{1.3}$$

where the *learning rate* η sets the step size. For the MSE loss the gradient has a clean closed form — with $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{w} \in \mathbb{R}^d$, and $\mathbf{y} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \ \in \ \mathbb{R}^d. \tag{1.4}$$

This iterative loop — predict, measure the loss, feel the gradient, step — is exactly what modern deep learning frameworks automate, at billion-parameter scale. When the dataset itself is huge we will not even use all of it per step: a small random *batch* gives a good-enough gradient. That refinement (stochastic gradient descent) matters enough to get its own treatment in Chapter 4.

1.6. Build it three times

Enough theory — let us implement linear regression, and let us do it in PyTorch. You have already built this in NumPy in earlier courses, so we will use it as an excuse to get comfortable with tensors, while still doing everything from scratch.

> **Coding hygiene**
>
> Two habits worth forming from the very first cell: **fix your seeds** so runs are reproducible, and **annotate your functions with types** — not critical for Python to run, but it makes code readable and debugging far easier.

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(6050)  # reproducible runs, always
```

```
<torch._C.Generator at 0x10fe8c3d0>
```

First, synthetic data. We control the ground truth — the true weights and bias — so we can check whether each method actually recovers them.

```python
def make_synthetic_data(
    weights: torch.Tensor,   # (d,) true weights
    bias: float,             # true bias
    n_samples: int,
    noise: float = 0.1,      # std of Gaussian noise
) -> tuple[torch.Tensor, torch.Tensor]:
    """y = X w + b + eps, with eps ~ N(0, noise^2)."""
    d = weights.shape[0]
    X = torch.randn(n_samples, d)              # (n, d) feature matrix
    y = X @ weights + bias                     # (n,)  clean targets
    y += noise * torch.randn(n_samples)        # add irreducible noise
    return X, y

true_w = torch.tensor([2.0, -3.4])
true_b = 4.2
X, y = make_synthetic_data(true_w, true_b, n_samples=200)
X.shape, y.shape   # (200, 2), (200,) - always check your shapes
```

```
(torch.Size([200, 2]), torch.Size([200]))
```

1.6.1. Take 1: the closed form

The normal equations, Equation 1.2, on the *augmented* matrix (the column of ones carries the bias):

```python
X_aug = torch.cat([X, torch.ones(X.shape[0], 1)], dim=1)   # (n, d+1)
w_ols = torch.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)   # solve, don't invert
print(f"closed form:      w = {w_ols[:2].numpy().round(3)},  b = {w_ols[2]:.3f}")
print(f"ground truth:     w = {true_w.numpy()},  b = {true_b}")
```

```
closed form:      w = [ 2.01  -3.402],  b = 4.196
ground truth:     w = [ 2.  -3.4],  b = 4.2
```

Two lines, and we recover the truth up to the noise floor. Remember what this cost us: $O(d^3)$, and the special grace of a linear model. Now the method that scales.

1.6.2. Take 2: gradient descent, from scratch

We write the model as a small class — a forward pass, a loss, manually derived gradients (Equation 1.4), and an update step. No autograd yet: we want to feel the mechanics once with our own hands.

```python
class LinearRegressionScratch:
    """Linear regression with manually derived gradients."""

    def __init__(self, input_size: int, lr: float = 0.1):
        self.w = 0.01 * torch.randn(input_size)   # small random start
        self.b = torch.zeros(1)
        self.lr = lr

    def forward(self, X: torch.Tensor) -> torch.Tensor:
        return X @ self.w + self.b                # (n,)

    @staticmethod
    def loss(y_hat: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        return ((y_hat - y) ** 2).mean()

    def step(self, X: torch.Tensor, y: torch.Tensor) -> float:
        y_hat = self.forward(X)                   # 1. predict
        loss = self.loss(y_hat, y)                # 2. measure
        err = y_hat - y                           # (n,) residuals
        grad_w = 2.0 * X.T @ err / len(y)         # 3. feel the slope  (d,)
        grad_b = 2.0 * err.mean()                 #    (bias grad = mean error)
        self.w -= self.lr * grad_w                # 4. step downhill
        self.b -= self.lr * grad_b
        return float(loss)

model = LinearRegressionScratch(input_size=2)
losses = [model.step(X, y) for _ in range(100)]
print(f"gradient descent: w = {model.w.numpy().round(3)},  b = {model.b.item():.3f}")
```

```
gradient descent: w = [ 2.01  -3.402],  b = 4.196
```

```python
plt.figure(figsize=(5.5, 3.2))
plt.plot(losses)
plt.xlabel("step")
plt.ylabel("MSE loss")
plt.yscale("log")
plt.tight_layout()
plt.show()
```

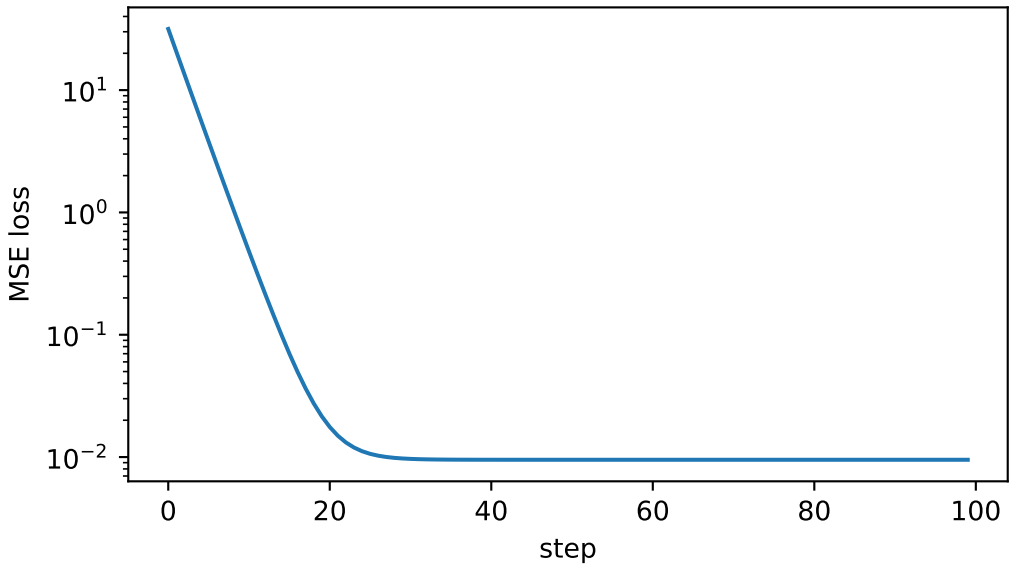

Figure 1.2.: The loss falls fast at first — steep slope underfoot — then flattens as we approach the valley floor set by the irreducible noise.

1.6.3. Take 3: the framework way

Finally, the idiom you will use for every model from here on: `nn.Linear` holds the knobs, autograd feels the slope for us, and the optimizer takes the step.

```
from torch import nn

net = nn.Linear(in_features=2, out_features=1)
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for _ in range(100):
    optimizer.zero_grad()               # clear old gradients - they accumulate!
    y_hat = net(X).squeeze(-1)          # (n, 1) -> (n)
    loss = loss_fn(y_hat, y)
    loss.backward()                     # autograd computes the gradient for us
    optimizer.step()

w_nn = net.weight.detach().squeeze()
b_nn = net.bias.item()
print(f"nn.Linear:        w = {w_nn.numpy().round(3)},  b = {b_nn:.3f}")
```

```
nn.Linear:        w = [ 2.01  -3.402],  b = 4.196
```

Three implementations, one answer:

```
grid = torch.linspace(X[:, 0].min(), X[:, 0].max(), 50)
line = lambda w, b: w[0] * grid + w[1] * X[:, 1].mean() + b

plt.figure(figsize=(5.5, 3.6))
plt.scatter(X[:, 0], y, s=12, alpha=0.4, label="data")
plt.plot(grid, line(w_ols, w_ols[2]), lw=3, label="closed form")
plt.plot(grid, line(model.w, model.b.item()), "--", lw=2, label="gradient descent")
plt.plot(grid, line(w_nn, b_nn), ":", lw=2, label="nn.Linear")
plt.xlabel("$x_1$"); plt.ylabel("$y$"); plt.legend()
plt.tight_layout()
plt.show()
```

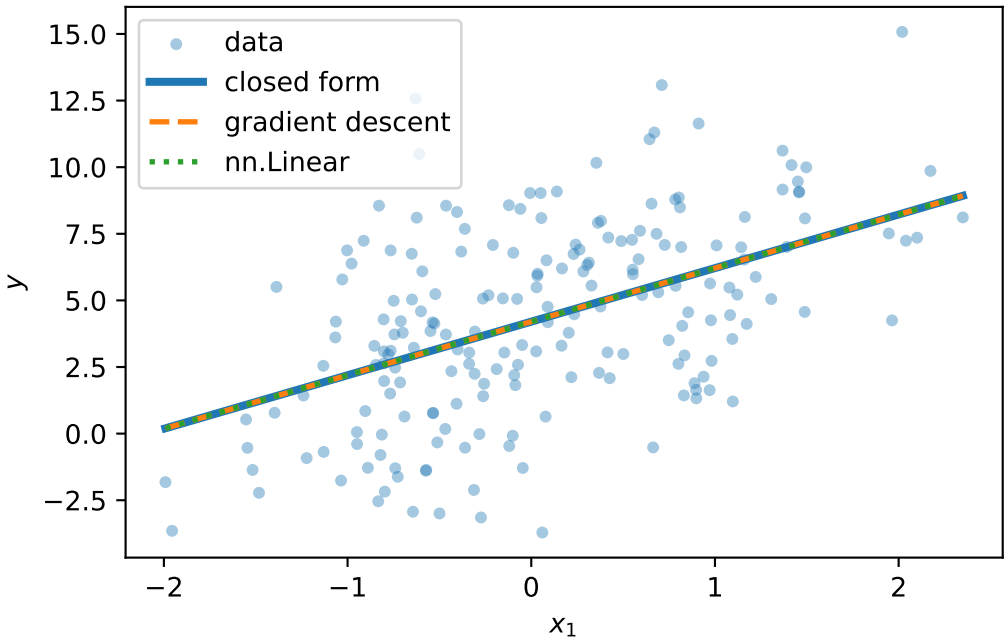

Figure 1.3.: All three methods find the same line (shown against feature x_1, second feature at its mean).

1.7. Why squared error? The maximum-likelihood view

The MSE is not an arbitrary choice. Assume the data really is generated by a linear process plus Gaussian noise:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Then the probability of seeing output y given input $\mathbf{x}$ is a Gaussian centered on the model's prediction. **Maximum likelihood estimation** asks: which parameters make the observed data

most likely? Take the log of the likelihood over all n samples and the Gaussian's exponent comes down:

$$\log \mathcal{L}(\mathbf{w}, b) = C - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^\top \mathbf{x}_i + b)\right)^2.$$

Maximizing the log-likelihood is *exactly* minimizing the sum of squared errors. The loss encodes an assumption about the noise. This is a pattern to remember: when we meet cross-entropy in Chapter 2, it will be the same story with a different distribution.

1.8. A first look at bias and variance

Our learned $\hat{\mathbf{w}}$ depends on the training data — and the training data is a random sample. Collect a fresh dataset and you get a different $\hat{\mathbf{w}}$. So the model's expected error on a new point decomposes into three parts:

$$\text{Err} = \underbrace{\left(\mathbb{E}[\hat{y}] - y\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[(\hat{y} - \mathbb{E}[\hat{y}])^2\right]}_{\text{Variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{Irreducible noise}}. \tag{1.5}$$

Bias measures how far the *average* prediction is from the truth, because of limiting assumptions like linearity — a simple model on complex data is systematically wrong (underfitting). **Variance** measures how much the prediction would change if we collected a fresh training set — the model's sensitivity to sampling noise; a complex model can fit the noise itself (overfitting). The **irreducible noise** is inherent to the data-generating process and cannot be learned away.

This is why we split data into training, validation, and test: the model sees the training set, the validation set referees the bias–variance trade-off, and the test set stays untouched until the end.

> ⚠ The U-shaped curve is a cartoon, not a law of nature
>
> The textbook picture — validation error falling then rising as complexity grows — is a helpful first mental model, and you should have it. But bias and variance are not a mechanical see-saw. More data shrinks variance (at a rate of roughly $1/n$, while leaving bias untouched); regularization, and inductive biases matched to the task, can buy capacity without exploding variance. Modern over-parameterized networks live in regimes where this cartoon bends in surprising ways — we will look at those pictures properly in Chapter 6.

1.8.1. Regularization, briefly

One lever we can pull right now: penalize complexity in the loss itself. **Ridge regression** adds an L_2 penalty,

$$\mathcal{L}_\lambda(\mathbf{w}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2, \qquad \hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + \lambda I)^{-1}\mathbf{X}^\top\mathbf{y},$$

nudging the model toward smaller, more stable weights — a little more bias for a lot less variance. Its cousin **lasso** uses the L_1 norm, $\lambda\|\mathbf{w}\|_1$, and does something qualitatively different: it drives the least informative weights to *exactly zero*, performing automatic feature selection.

Let us see that difference, not just assert it. We build a problem designed to punish an unregularized model: 20 features of which only 5 matter, noisy targets, and — the cruel part — barely more samples than parameters. This is exactly the high-dimensional regime where regularization shines:

```
from sklearn.linear_model import Lasso   # coordinate descent for L1 (no closed form)

n, d = 25, 20                            # barely more samples than knobs!
w_true = torch.zeros(d)
w_true[:5] = torch.tensor([3.0, -2.0, 1.5, 2.5, -1.0])   # only 5 real features
Xh, yh = make_synthetic_data(w_true, bias=0.0, n_samples=n, noise=2.0)

# OLS and ridge in closed form
w_ols_h = torch.linalg.solve(Xh.T @ Xh, Xh.T @ yh)
lam = 10.0
w_ridge = torch.linalg.solve(Xh.T @ Xh + lam * torch.eye(d), Xh.T @ yh)
w_lasso = torch.tensor(
    Lasso(alpha=0.4).fit(Xh.numpy(), yh.numpy()).coef_, dtype=torch.float32
)

def report(name: str, w_hat: torch.Tensor) -> None:
    err = ((w_hat - w_true) ** 2).mean()
    zeros = int((w_hat.abs() < 1e-3).sum())
    print(f"{name:>6}:  weight MSE = {err:.4f}   near-zero weights = {zeros}/20")

report("OLS", w_ols_h)
report("ridge", w_ridge)
report("lasso", w_lasso)
```

```
   OLS:  weight MSE = 1.2347   near-zero weights = 0/20
 ridge:  weight MSE = 0.4400   near-zero weights = 0/20
 lasso:  weight MSE = 0.3489   near-zero weights = 12/20
```

OLS, with all its freedom and almost no data to discipline it, assigns large, confident, *wrong* weights to the 15 noise features — pure variance. Ridge shrinks everything and cuts the damage several-fold, but keeps every feature alive. Lasso does the one thing the others cannot: it identifies the noise features and sets them *exactly* to zero — a sparse, more interpretable model that has effectively performed feature selection for us. (Rerun this cell with `n = 100` and watch OLS become respectable again: variance decays like $1/n$.)

1.9. Okay, so — what did we just build?

We taught a computer a function from examples, completely:

1. **A model family** — hyperplanes, parameterized by knobs $\mathbf{w}, b$ (with the augmentation trick to fold b into $\mathbf{w}$).
2. **A loss** — MSE, which is maximum likelihood under Gaussian noise; the loss is how we tell the model how bad it is.
3. **An optimizer** — the exact projection when linearity permits, and gradient descent (feel the slope, step downhill) when it does not — which is always, from here on.
4. **The generalization lens** — bias vs. variance, the data splits that referee them, and regularization as a first counter-measure to overfitting.

And one reframing that will carry the whole book: linear regression is a single neuron with the identity activation,

$$\underbrace{y = \mathbf{w}^\top \mathbf{x} + b}_{\text{this chapter}} \quad \xrightarrow{\text{add a nonlinearity}} \quad \underbrace{y = \sigma(\mathbf{w}^\top \mathbf{x} + b)}_{\text{a neuron}}.$$

First, in Chapter 2, that squashing function turns our regressor into a classifier. Then, in Chapter 3, we stack neurons — and discover why the nonlinearity is not optional.

Exercises

1. **(Pencil.)** Derive the normal equations Equation 1.2 by expanding $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$, taking the gradient with respect to $\mathbf{w}$, and setting it to zero. Where exactly does the assumption that $\mathbf{X}^\top \mathbf{X}$ is invertible enter?
2. **(Pencil.)** Repeat the derivation with the ridge penalty $\lambda \|\mathbf{w}\|_2^2$ and show the solution becomes $(\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}$. Why does the λI term guarantee invertibility for any $\lambda > 0$?
3. **(Code.)** In `make_synthetic_data`, raise `noise` to 1.0 and rerun all three implementations 10 times with different seeds. How much do the learned weights vary across runs? Which term of Equation 1.5 are you watching?
4. **(Code.)** Set the learning rate in `LinearRegressionScratch` to 1.0 and rerun. Describe what the loss curve does, and explain it using the blindfolded-descent picture.
5. **(Code.)** In the ridge/lasso experiment, sweep $\lambda \in \{0.01, 0.1, 1, 10, 100\}$ for ridge and plot weight MSE against λ. Where is the sweet spot, and what happens at the extremes? Connect your answer to bias and variance.

2. Linear Models that Classify: Logistic and Softmax

Regression predicts numbers; most of what the world wants predicted is *categories* — cat or dog, spam or not, which of ten digits. This chapter converts the linear machinery of Chapter 1 into a classifier without discarding a single idea: the same weighted sums, the same maximum-likelihood recipe for the loss, the same gradient descent. What changes is one squashing function at the output, and it changes less than you might think.

2.1. What actually changes

In regression, $y \in \mathbb{R}$ and the Gaussian noise model handed us the MSE (Chapter 1). In classification, y is a label from $\{0, \dots, K-1\}$. Could we just regress onto the labels with MSE anyway? We could, and it fails for a principled reason: **the loss is a noise model, and Gaussian is the wrong model for a coin flip.** A binary outcome deviates from its probability like a Bernoulli variable, not like additive Gaussian noise → maximum likelihood demands a different loss. Everything in this chapter falls out of taking that seriously.

What we need is two things:

1. a way to turn the model's raw score (its **logit**, $o \in \mathbb{R}$) into a probability,
2. a loss that measures how wrong a *probability* is.

2.2. Binary classification

2.2.1. From score to probability: the sigmoid

Our linear model produces $o = \mathbf{w}^\top \mathbf{x} + b$, an unbounded score. The **sigmoid** squashes it into a probability:

$$\sigma(o) = \frac{1}{1 + e^{-o}}. \tag{2.1}$$

```
import torch
import matplotlib.pyplot as plt

o = torch.linspace(-6, 6, 300)
plt.figure(figsize=(5.8, 3))
plt.plot(o, torch.sigmoid(o), color="#232D4B", lw=2.2)
plt.axhline(0.5, ls=":", color="#B8B8A8"); plt.axvline(0, ls=":", color="#B8B8A8")
plt.annotate("decision boundary\n$o = 0$", xy=(0, 0.5), xytext=(1.6, 0.32),
             color="#E57200", arrowprops=dict(arrowstyle="-|>", color="#E57200"))
plt.xlabel(r"logit $o = \mathbf{w}^\top \mathbf{x} + b$")
plt.ylabel(r"$\hat{p} = \sigma(o)$")
plt.tight_layout(); plt.show()
```

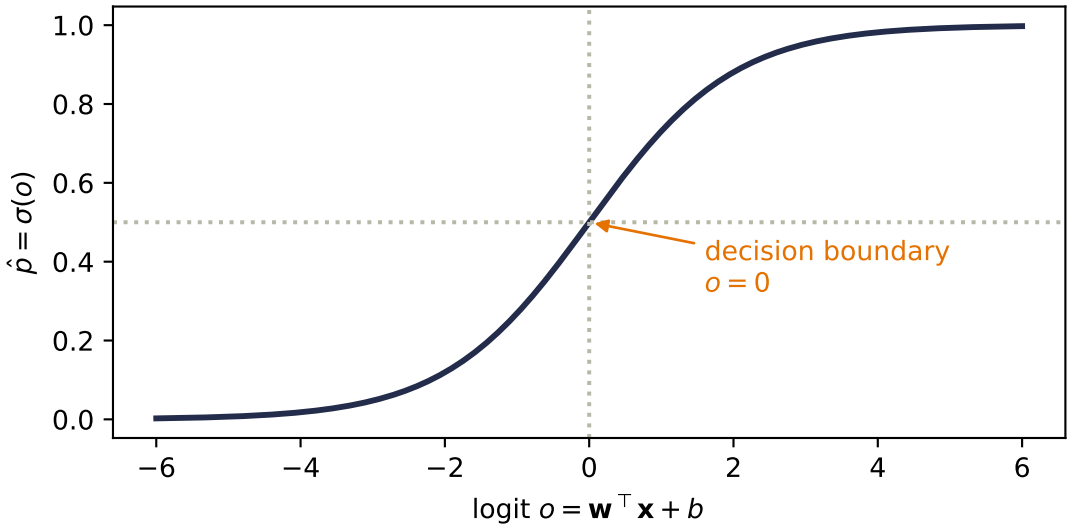

Figure 2.1.: The sigmoid maps any real score to $(0, 1)$, crossing $\frac{1}{2}$ exactly at $o = 0$. The model's uncertainty lives in the middle; its confident regions saturate at the tails.

Notice what did *not* change: the decision boundary $\hat{p} = \frac{1}{2}$ is exactly $o = 0$, i.e. $\mathbf{w}^\top \mathbf{x} + b = 0$ — the same hyperplane geometry from Chapter 1, with $\mathbf{w}$ still the template the input is scored against. The sigmoid does not bend the boundary; it grades our confidence on either side of it.

2.2.2. The loss: cross-entropy from maximum likelihood

Run the maximum-likelihood recipe with the correct noise model. If the label is a coin flip with $P(y{=}1 \mid \mathbf{x}) = \hat{p}$, the likelihood of the dataset is $\prod_i \hat{p}_i^{y_i}(1-\hat{p}_i)^{1-y_i}$; taking the negative log gives the **binary cross-entropy**:

$$\mathcal{L}_{\text{BCE}} = -\big[\, y \log \hat{p} + (1-y) \log(1-\hat{p}) \,\big]. \tag{2.2}$$

Read it as *surprise*: if the true label is 1, the loss is $-\log \hat{p}$ — small when the model believed in the outcome, exploding as the model's belief goes to zero. A confident wrong answer is punished without mercy, which is exactly the pressure a probability deserves.

2.2.3. The beautiful gradient

Chain the sigmoid into the cross-entropy and differentiate with respect to the logit, and something remarkable drops out (Exercise 1 walks you through it):

$$\frac{\partial \mathcal{L}}{\partial o} = \hat{p} - y. \tag{2.3}$$

The gradient is *literally the prediction error*, the same form MSE gave us for regression. All the logs and exponentials annihilate each other. This is not luck, and it is not the last time you will see it.

2.3. Multiclass classification: softmax

With K classes, run K templates at once: $\mathbf{W} \in \mathbb{R}^{K \times d}$ produces a score vector $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, one logit per class. To turn K scores into a probability distribution, exponentiate (making everything positive) and normalize:

$$\hat{y}_j = \mathrm{softmax}(\mathbf{o})_j = \frac{e^{o_j}}{\sum_{k=1}^{K} e^{o_k}}. \tag{2.4}$$

Softmax has three properties worth committing to memory: every output is positive, the outputs sum to one, and — less obviously — it is **invariant to constant shifts**: adding the same c to every logit changes nothing, because $e^{o_j + c} = e^c e^{o_j}$ and the e^c cancels top and bottom. Only the *differences* between scores matter.

```
logits = torch.tensor([2.0, 0.5, -1.0, 1.0])
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9), sharey=True)
for ax, shift in [(axes[0], 0.0), (axes[1], 100.0)]:
    p = torch.softmax(logits + shift, dim=0)
    ax.bar(range(4), p, color="#5379AA", width=0.55)
    for j, v in enumerate(p):
        ax.text(j, v + 0.015, f"{v:.2f}", ha="center", fontsize=9)
    ax.set_xticks(range(4))
    ax.set_xticklabels([f"$o_{j}$ = {v:+.1f}" for j, v in enumerate(logits + shift)],
                       fontsize=8)
    ax.set_title("original logits" if shift == 0 else "all logits + 100")
axes[0].set_ylabel("probability")
plt.tight_layout(); plt.show()
```

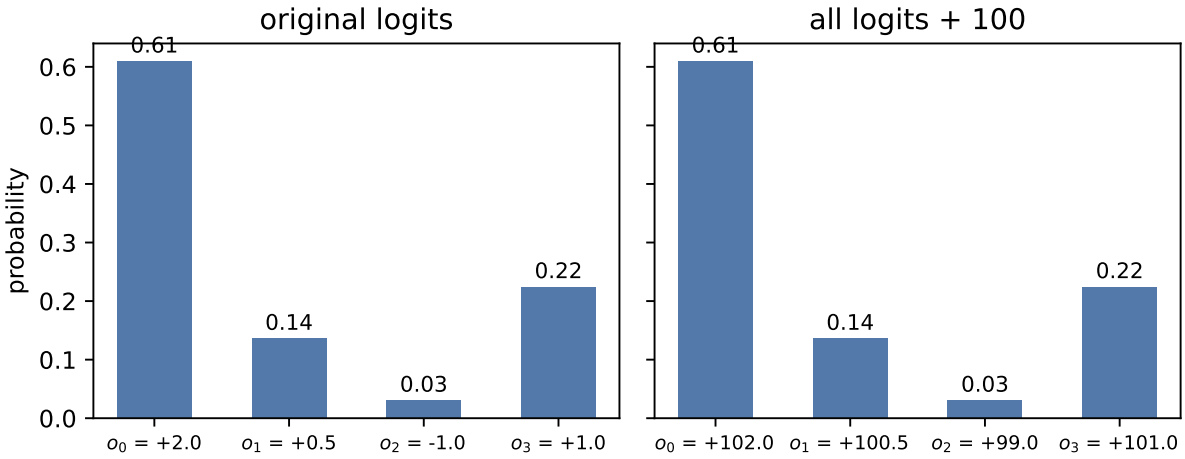

Figure 2.2.: Softmax turns scores into a distribution. Shifting all logits by a constant (right) changes the probabilities not at all — only score differences carry information.

Remember this machine: scores → weights

Step back from classification for a moment. What softmax really is: a differentiable device that takes any list of scores and returns **positive weights that sum to one**, favoring the large scores without silencing the small ones. Today the scores are class logits. In Part IV, the scores will be *similarities between a query and a set of keys*, and this exact same machine will turn them into **attention weights** (Chapter 13). One normalizer, the whole book long.

2.3.1. Cross-entropy, multiclass edition

With one-hot labels $\mathbf{y}$ (all zeros except a 1 at the true class), the cross-entropy is

$$\mathcal{L}_{\text{CE}} = -\sum_{j=1}^{K} y_j \log \hat{y}_j = -\log \hat{y}_{\text{true class}}, \tag{2.5}$$

the surprise at the true class, again. And the gradient repeats its trick:

$$\frac{\partial \mathcal{L}}{\partial o_j} = \hat{y}_j - y_j, \tag{2.6}$$

prediction minus target, one more time. This recurrence is no coincidence — sigmoid/BCE and softmax/CE are both members of the exponential family paired with their natural losses, and that pairing always collapses the gradient to the error.

> **The information-theory reading**
>
> Entropy $H[P] = -\sum_j p_j \log p_j$ measures the uncertainty in a distribution; cross-entropy $H(P,Q) = -\sum_j p_j \log q_j$ is the cost of encoding outcomes from P using codes built for Q. Minimizing our loss is minimizing that encoding cost — equivalently, maximizing likelihood, equivalently driving the KL divergence from truth to prediction toward zero. Three vocabularies, one objective.

2.3.2. One numerical landmine

Computing e^{o_j} overflows for logits as small as a few hundred. The fix exploits shift invariance: subtract the max logit before exponentiating (the *log-sum-exp trick*), which changes nothing mathematically and everything numerically. We will use it in the code below; the deeper story of floating-point limits lives in Appendix D.

2.4. Build it: three classes, from scratch

The full classifier, with the pieces named: K=3 templates, stable softmax, cross-entropy, and — because Equation 2.6 is so clean — the *manually derived* gradient, no autograd needed. On three Gaussian blobs:

```
torch.manual_seed(6050)
centers = torch.tensor([[-2.0, 0.0], [2.0, 0.0], [0.0, 2.5]])
X = torch.cat([c + 0.7 * torch.randn(150, 2) for c in centers])   # (450, 2)
y = torch.arange(3).repeat_interleave(150)                        # (450,)
Y = torch.nn.functional.one_hot(y, 3).float()                     # (450, 3)

def stable_softmax(logits: torch.Tensor) -> torch.Tensor:
    z = logits - logits.max(dim=-1, keepdim=True).values   # shift invariance at work
    e = torch.exp(z)
    return e / e.sum(dim=-1, keepdim=True)

W, b = torch.zeros(3, 2), torch.zeros(3)
for step in range(400):
    P = stable_softmax(X @ W.T + b)          # (450, 3) predicted probabilities
    G = (P - Y) / len(X)                     # the beautiful gradient, eq. (6)
    W -= 1.0 * G.T @ X                       # blame out x signal in
    b -= 1.0 * G.sum(0)

ce = -(Y * torch.log(P)).sum(1).mean()
acc = ((X @ W.T + b).argmax(1) == y).float().mean()
print(f"cross-entropy {ce:.3f}   accuracy {acc:.1%}")
```

```
cross-entropy 0.033   accuracy 98.9%
```

```
import numpy as np

gx, gy = np.meshgrid(np.linspace(-4.5, 4.5, 300), np.linspace(-2.5, 5, 300))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)
Pg = stable_softmax(grid @ W.T + b)
cls = Pg.argmax(1).reshape(gx.shape).numpy()
conf = Pg.max(1).values.reshape(gx.shape).numpy()

plt.figure(figsize=(6, 4.4))
plt.contourf(gx, gy, cls, levels=[-0.5, 0.5, 1.5, 2.5],
             colors=["#DCE6F2", "#FDE3C8", "#E3EEE3"])
plt.contourf(gx, gy, 1 - conf, levels=12, cmap="Greys",
             alpha=0.25, antialiased=True)
for k, color in enumerate(["#232D4B", "#E57200", "#6B8E6B"]):
    plt.scatter(X[y == k, 0], X[y == k, 1], s=8, c=color, label=f"class {k}")
plt.xlabel("$x_1$"); plt.ylabel("$x_2$"); plt.legend(loc="lower right")
plt.tight_layout(); plt.show()
```

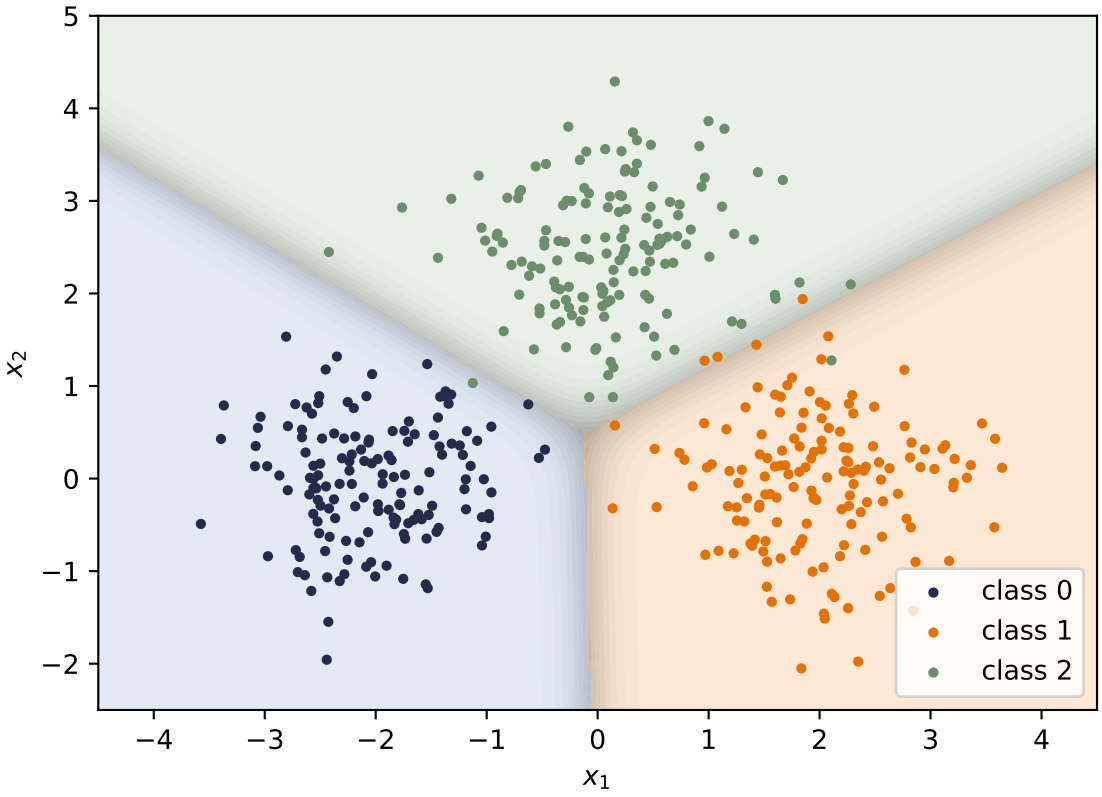

Figure 2.3.: Three templates, three linear boundaries. Color shows the predicted class; shading shows confidence (the max softmax probability) — crisp deep in each region, washed out along the boundaries where classes compete.

The framework version is two lines, with one trap worth a warning:

```
loss_fn = torch.nn.CrossEntropyLoss()
print(f"ours: {ce:.6f}   torch: {loss_fn(X @ W.T + b, y):.6f}")
```

```
ours: 0.033438   torch: 0.033425
```

> ⚠ The most common classification bug
>
> `nn.CrossEntropyLoss` takes **raw logits**, not softmax outputs — it applies (log-)softmax internally, with the stability trick built in. Feed it probabilities and your model still trains, just badly, with no error message. If a classifier is mysteriously mediocre, check this first.

2.5. Okay, so — classification is regression plus a normalizer

1. **The geometry survived**: logits are still template similarities $\mathbf{w}^\top \mathbf{x} + b$, boundaries are still hyperplanes. Sigmoid and softmax grade confidence; they do not bend anything.
2. **The loss came from the noise model**, exactly as in Chapter 1: Bernoulli → BCE, categorical → cross-entropy, both read as *surprise at the truth*.
3. **The gradient is the error**, $\hat{y} - y$, both times — the exponential-family pairing at work.
4. **Softmax is a scores-to-weights machine**, shift-invariant, numerically tamed by log-sum-exp — and it will return, unchanged, as the normalizer of attention.

Next, we let the templates themselves be built out of other templates: Chapter 3 stacks these linear units — and discovers why a bend between them is not optional.

Exercises

1. **(Pencil.)** Derive Equation 2.3: write $\mathcal{L}_{\text{BCE}}$ as a function of o through $\hat{p} = \sigma(o)$, use $\sigma'(o) = \sigma(o)(1-\sigma(o))$, and simplify. Which factors cancel, and why is the result independent of the sigmoid's saturation?
2. **(Pencil.)** Prove softmax's shift invariance from Equation 2.4, then explain why subtracting the max logit makes the computation overflow-proof: what is the largest value ever exponentiated?
3. **(Code.)** Temperature: replace `logits` with `logits / T` in the softmax figure for $T \in \{0.5, 1, 2, 10\}$. Describe what happens to the distribution at both extremes. (Keep this dial in mind — it returns when we scale attention scores in Part IV.)
4. **(Code.)** Train the blobs classifier with MSE on one-hot targets instead of cross-entropy (same manual loop; the gradient changes). Compare accuracy and the confidence shading near boundaries. What does the wrong noise model cost?
5. **(Code.)** Move the three blob centers closer together (`0.7 → 1.2` noise, or centers at distance 1) and retrain. Where does the confidence shading collapse, and what does the cross-entropy value tell you compared to the accuracy?

3. Nonlinearity and the MLP

Our linear models can regress (Chapter 1) and classify (Chapter 2), and it is tempting to think we could get more power by simply stacking them: feed the output of one linear layer into another. Watch what happens:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\,\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{\text{new}}\,\mathbf{x} + \mathbf{b}_{\text{new}}. \tag{3.1}$$

The stack *collapses*. Two linear layers, or twenty, are exactly one linear layer with different knobs → no amount of stacking buys any new expressive power. To learn the patterns of the real world we need a magic ingredient: **nonlinearity**. This chapter is about what one small bend does to everything.

3.1. The tiny dataset that embarrassed a field

You do not need real-world data to expose the linear ceiling. Four points suffice. The XOR function outputs 1 exactly when its two binary inputs *differ*:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot the four points and try to separate the 1s from the 0s with a single line. The positive examples sit on one diagonal, the negatives on the other; any line you draw puts at least one point on the wrong side. A linear classifier cannot represent this function, no matter how it is trained.

> **i** A historical scar
>
> This is not a toy observation. The XOR limitation of single-layer perceptrons, publicized in 1969, helped freeze neural-network research for years — part of the first "AI winter." The thaw came with multi-layer networks and backpropagation (Chapter 5), which is to say: with the contents of this chapter and the next two.

Let us confirm the ceiling empirically, with the tools we already have:

```
import torch
from torch import nn

torch.manual_seed(6050)
X_xor = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y_xor = torch.tensor([0., 1., 1., 0.])

def train(model, X, y, steps=4000, lr=0.5):
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        loss = nn.functional.binary_cross_entropy_with_logits(model(X).squeeze(-1), y)
        loss.backward()
        opt.step()
    return model

linear = train(nn.Linear(2, 1), X_xor, y_xor)
mlp = train(nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1)),
            X_xor, y_xor)

for name, model in [("linear", linear), ("MLP (4 hidden)", mlp)]:
    acc = ((model(X_xor).squeeze(-1) > 0).float() == y_xor).float().mean()
    print(f"{name:>15}: accuracy {acc:.0%}")
```

```
         linear: accuracy 25%
 MLP (4 hidden): accuracy 100%
```

The linear model lands at 25%: *worse than guessing*, because XOR is adversarial to lines. The little network with one bend in it is perfect. The rest of the chapter explains what that bend actually buys.

```
import numpy as np
import matplotlib.pyplot as plt

gx, gy = np.meshgrid(np.linspace(-0.6, 1.6, 220), np.linspace(-0.6, 1.6, 220))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3.4), sharey=True)
for ax, model, title in [(axes[0], linear, "linear model"),
                         (axes[1], mlp, "MLP with ReLU")]:
    with torch.no_grad():
        Z = (model(grid).squeeze(-1) > 0).float().reshape(gx.shape)
    ax.contourf(gx, gy, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    for cls, marker, color in [(0, "o", "#232D4B"), (1, "s", "#E57200")]:
```

```
        pts = X_xor[y_xor == cls]
        ax.scatter(pts[:, 0], pts[:, 1], marker=marker, s=120, c=color,
                   edgecolors="white", linewidths=1.5, zorder=3,
                   label=f"class {cls}")
    ax.set_title(title); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$"); axes[0].legend(loc="center right")
plt.tight_layout(); plt.show()
```

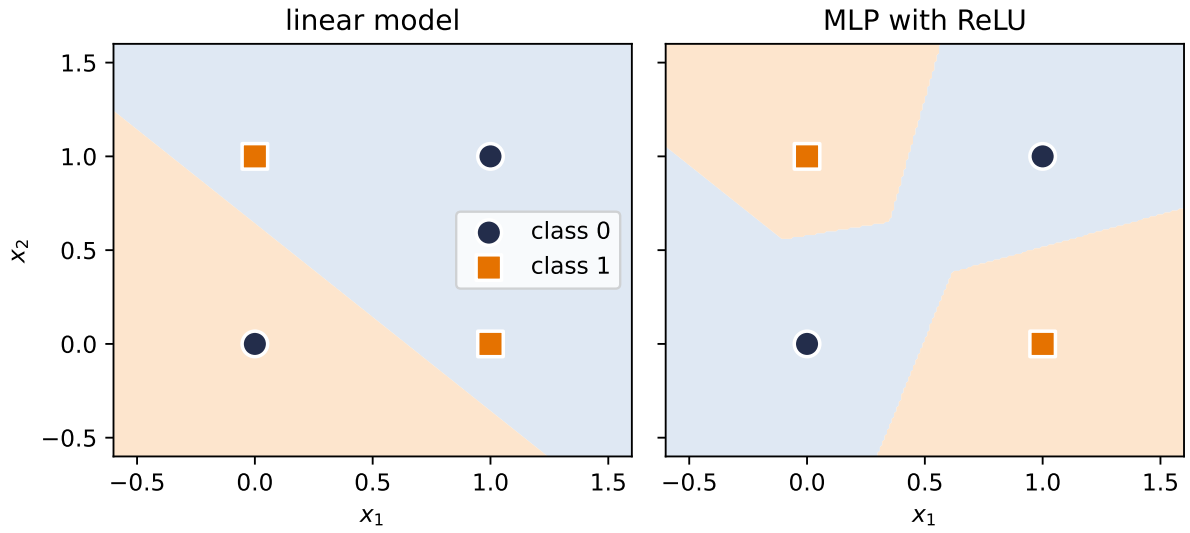

Figure 3.1.: Left: no line separates XOR — whatever the linear model learns, a diagonal pair is misclassified. Right: the trained 4-hidden-unit MLP carves the plane so that both diagonals get their own region.

3.2. The neuron: a threshold with a bend

The artificial neuron takes its inspiration from biology: a neuron receives signals, and it *fires* only when the accumulated stimulus crosses a threshold. The modern, computationally friendly version of that thresholding is the **rectified linear unit**:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad f(\mathbf{x}) = \mathrm{ReLU}(\mathbf{w}^\top \mathbf{x} + b). \tag{3.2}$$

The weighted input $\mathbf{w}^\top \mathbf{x} + b$ is the stimulus — and recall from Chapter 1 that it is a *similarity score* against the template $\mathbf{w}$. Below threshold, the neuron is silent (output 0); above it, the neuron fires with intensity proportional to the stimulus.

In one dimension a single neuron turns a line into a **hinge**, and in two dimensions it draws a line through the plane: an OFF region and an ON region, with $\mathbf{w}$ perpendicular to the boundary. That boundary is the fundamental unit of computation in everything that follows.

(What about the sharp corner at $z = 0$, where the derivative is undefined? In practice, never a problem: libraries assign it 0, and the chance of a floating-point number landing exactly on 0 is essentially nil. The gradient story of this choice — why this open-or-shut gate is *the* reason deep networks train — is told in Chapter 5.)

3.3. Layers of hinges: bumps and polytopes

One neuron makes one boundary. A layer of k neurons makes k boundaries, and their intersections partition the input space into cells — each cell a **convex polytope**, and within each cell the layer computes a plain linear function. With d-dimensional inputs, k neurons can create on the order of k^d regions.

The simplest constructive example lives in 1D. Take two hinges and subtract:

```
xs = torch.linspace(-3, 5, 400)
h1 = torch.relu(0.67 * xs + 1)
h2 = torch.relu(0.8 * xs - 0.4)
bump = h1 - 1.25 * h2

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3))
axes[0].plot(xs, h1, color="#5379AA", label=r"$h_1 = \mathrm{ReLU}(0.67x + 1)$")
axes[0].plot(xs, h2, color="#6B8E6B", label=r"$h_2 = \mathrm{ReLU}(0.8x - 0.4)$")
axes[0].set_title("two hidden units"); axes[0].legend(fontsize=8)
axes[1].plot(xs, bump, color="#E57200", lw=2.5, label=r"$y = h_1 - 1.25\,h_2$")
axes[1].set_title("their combination: a bump"); axes[1].legend(fontsize=9)
for ax in axes: ax.set_xlabel("$x$")
plt.tight_layout(); plt.show()
```

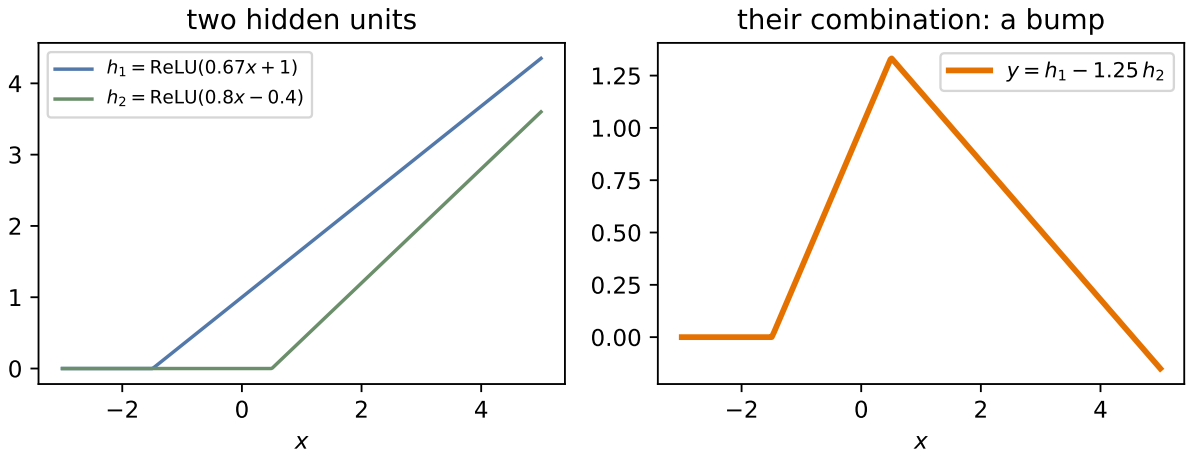

Figure 3.2.: The lecture's construction, exactly: two hinges (h_1, h_2) and one subtraction make a bump. Bumps are the brush strokes of function approximation.

Follow the pieces: before the first hinge, both units are silent and the output is zero. Between the hinges, only h_1 is on, so the output climbs. After the second hinge, h_2's negative contribution takes over and the output descends back to zero. Three linear pieces → one bump.

3.4. The universal approximation theorem

Bumps are the whole secret. If you can make one bump, you can make many, at any position and height → with enough hidden units you can *tile* any continuous function, the way an artist paints a shape with brush strokes. That is the intuition behind the **universal approximation theorem**: a network with a single hidden layer can, in principle, approximate any continuous function on a bounded region to any desired accuracy.

Watch the theorem materialize as width grows:

```
xs1 = torch.linspace(-3, 3, 200)[:, None]
target = torch.sin(2 * xs1) + 0.5 * torch.sign(xs1)

def fit(width: int, steps: int = 3000, lr: float = 0.05):
    torch.manual_seed(6050)
    net = nn.Sequential(nn.Linear(1, width), nn.ReLU(), nn.Linear(width, 1))
    opt = torch.optim.SGD(net.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        ((net(xs1) - target) ** 2).mean().backward()
        opt.step()
    with torch.no_grad():
        return net(xs1).squeeze(-1)

plt.figure(figsize=(6.6, 3.6))
plt.plot(xs1, target, color="#232D4B", lw=2.5, label="target")
for width, color in [(2, "#B8B8A8"), (8, "#5379AA"), (64, "#E57200")]:
    plt.plot(xs1, fit(width), color=color, lw=1.6, label=f"width {width}")
plt.xlabel("$x$"); plt.legend(); plt.tight_layout(); plt.show()
```

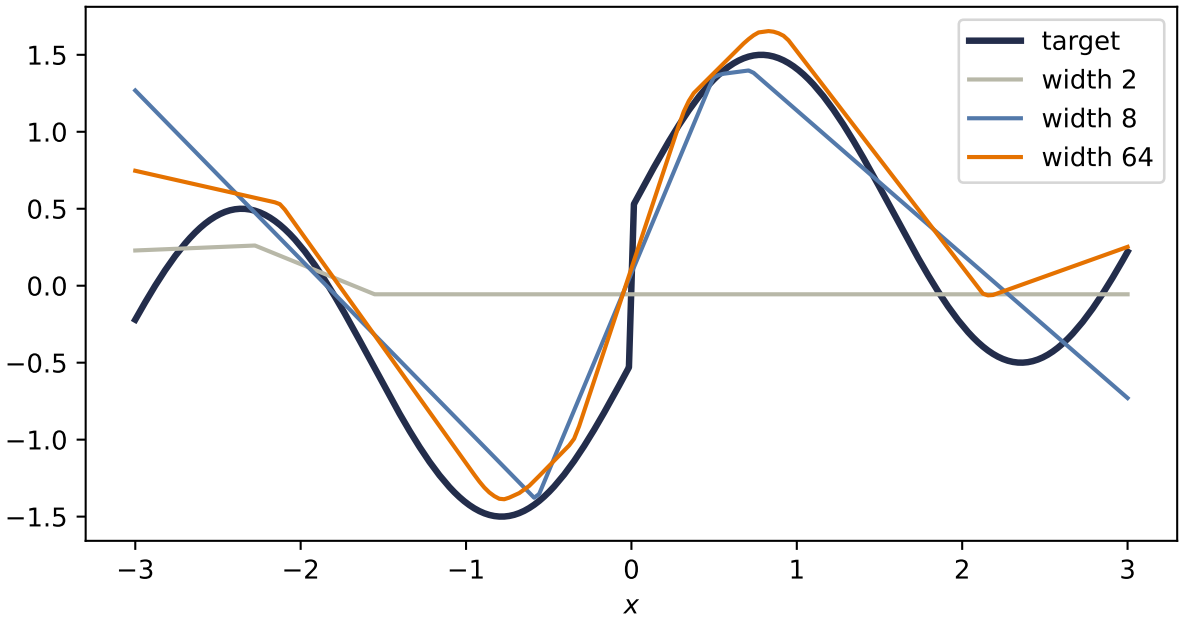

Figure 3.3.: One hidden layer approximating a wiggly target with a jump. Width 2 manages a single hinge-pair; width 8 catches the shape; width 64 traces it closely — except at the discontinuity, which no finite (or infinite) width fully conquers: the theorem promises continuous functions only.

Now read the theorem's fine print, because it is where beginners over-promise:

⚠ What the theorem does *not* say

1. It does not tell you **how to find the weights** — existence of a good network is not a training algorithm. Finding the weights is the job of the loss and the optimizer, and nothing guarantees gradient descent lands on the theorem's solution.
2. It does not say the network is **small**. Tiling a complicated function with one-layer bumps can require astronomically many units.
3. It speaks of **continuous functions on bounded regions** — note our jump refusing to be captured — and says nothing about how the fit behaves *between* or *beyond* your samples. Approximation is not generalization; that distinction gets its own chapter (Chapter 6).

3.5. Why depth: boundaries that bend

If one hidden layer suffices in principle, why is the field called *deep* learning? Efficiency, and geometry. A second hidden layer receives, as its inputs, the piecewise-linear outputs of the first → its boundaries are no longer straight lines. They *bend* wherever the first layer's boundaries cut the space. Boundaries composed of boundaries: each layer folds the space the previous layer drew.

```
torch.manual_seed(6050)
net2 = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 8), nn.ReLU())

gx2, gy2 = np.meshgrid(np.linspace(-2, 2, 500), np.linspace(-2, 2, 500))
G = torch.tensor(np.stack([gx2.ravel(), gy2.ravel()], 1), dtype=torch.float32)
with torch.no_grad():
    h1_ = torch.relu(net2[0](G))
    h2_ = torch.relu(net2[2](h1_))
    pattern = torch.cat([(h1_ > 0), (h2_ > 0)], 1).float()
    code = (pattern @ (2 ** torch.arange(16).float())).reshape(gx2.shape)

plt.figure(figsize=(5.4, 4.6))
plt.imshow(code, extent=[-2, 2, -2, 2], origin="lower", cmap="tab20", alpha=0.85)
plt.xlabel("$x_1$"); plt.ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

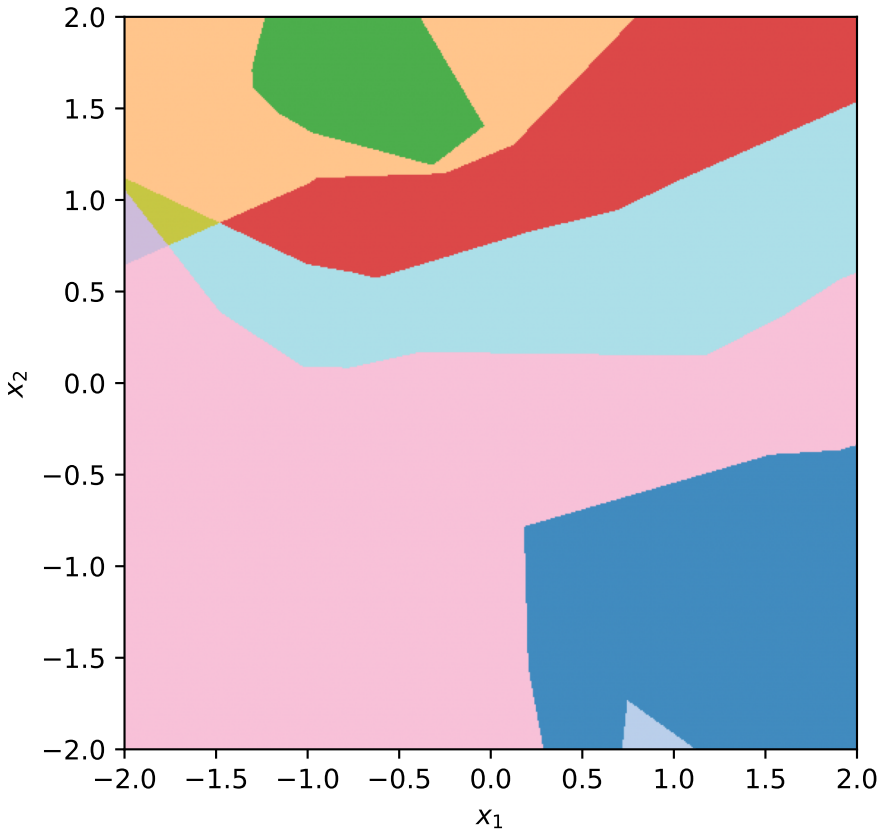

Figure 3.4.: The plane, colored by which ReLUs are on/off in a random 2-layer network ($2 \to 8 \to 8$): every cell is a region where the network is exactly linear. First-layer boundaries are straight; second-layer boundaries visibly bend where they cross them.

This bending compounds: theory shows the number of linear regions grows **exponentially with**

depth but only polynomially with width. A deep, narrow network can carve a partition that a shallow network would need exponentially many units to match. That is the economic argument for depth — the same expressive budget, spent far more efficiently.

3.6. Your first real MLP

Time to build the thing properly, the way the live session does: as a class.

> Coding hygiene: the house and its foundation
>
> Every model you write from now on subclasses `nn.Module`, and the first line of your `__init__` is always `super().__init__()`. Think of it as pouring the foundation before building the house: it is what wires up parameter tracking, `.parameters()`, and everything the optimizer and autograd need. Forget it and PyTorch fails loudly — the one favor a framework can do you.

```
class MLP(nn.Module):
    """A multilayer perceptron: linear layers with bends between them."""

    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int):
        super().__init__()                      # the foundation
        self.hidden = nn.Linear(input_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim, output_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.out(torch.relu(self.hidden(x)))
```

And a problem worthy of it: two interleaved moons, the classic shape no line can separate well. Same training loop as always — the model is the only thing that changed:

```
torch.manual_seed(6050)
n = 200
t = torch.rand(n) * torch.pi
moon1 = torch.stack([torch.cos(t), torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
moon2 = torch.stack([1 - torch.cos(t), 0.4 - torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
X_m = torch.cat([moon1, moon2])
y_m = torch.cat([torch.zeros(n), torch.ones(n)])

torch.manual_seed(6050)
lin_m = train(nn.Linear(2, 1), X_m, y_m, steps=3000, lr=0.8)
torch.manual_seed(6050)
mlp_m = train(MLP(2, 16, 1), X_m, y_m, steps=3000, lr=0.8)

gx3, gy3 = np.meshgrid(np.linspace(-1.6, 2.6, 240), np.linspace(-1.1, 1.6, 240))
```

```
grid3 = torch.tensor(np.stack([gx3.ravel(), gy3.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.6, 3.3), sharey=True)
for ax, model, title in [(axes[0], lin_m, "linear"), (axes[1], mlp_m, "MLP (16 hidden)")]:
    with torch.no_grad():
        acc = ((model(X_m).squeeze(-1) > 0).float() == y_m).float().mean()
        Z = (model(grid3).squeeze(-1) > 0).float().reshape(gx3.shape)
    ax.contourf(gx3, gy3, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    ax.scatter(X_m[y_m == 0, 0], X_m[y_m == 0, 1], s=8, c="#232D4B")
    ax.scatter(X_m[y_m == 1, 0], X_m[y_m == 1, 1], s=8, c="#E57200")
    ax.set_title(f"{title}: {acc:.1%}"); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

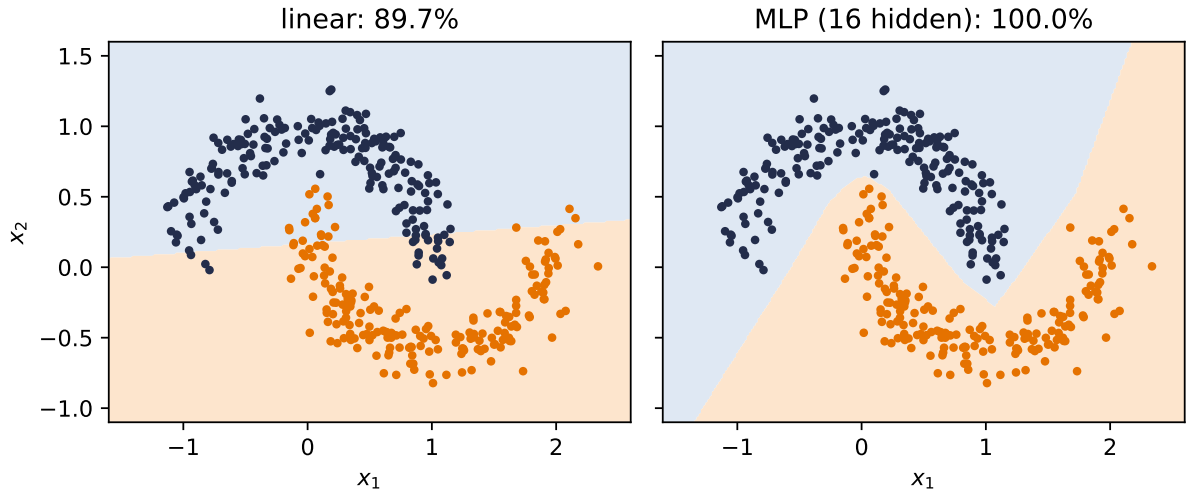

Figure 3.5.: Two moons: the linear model does what it can with one line (about 90% here); the MLP bends its boundary through the gap and separates the classes completely.

One honest footnote on the XOR network: in principle *two* hidden units suffice, but such minimal networks are temperamental — across random seeds they frequently get stuck at 50% accuracy in poor local minima (you will verify this in Exercise 3, and the noisy escape story of Chapter 4 is part of the cure). A few spare units cost little and buy reliability.

3.7. A cautionary tale, and a look ahead

There is a seductive story about what the layers of an MLP learn: the first layer detects edges, the next combines them into shapes, the next into objects — a tidy hierarchy of increasingly abstract features. It is the *idealized* story, and for fully-connected networks on raw images it is mostly **not what happens**. A fully-connected layer sees its input as one long, structureless

vector; it has no notion that pixel potentially relates to its neighbor. Nothing in the architecture encourages the tidy hierarchy, and with every pixel connected to every unit, the parameter count explodes long before the hierarchy emerges.

We now hold real expressive power — networks that can, in principle, represent almost anything. Chapter 4 trains them at scale and Chapter 5 supplies their gradients. And then Chapter 6 asks this chapter's uncomfortable question: what happens when all that flexibility meets data it does not understand the *structure* of? The answer — watch an MLP fail, in pictures, then fix it by building knowledge into the architecture — is the turn the whole book pivots on.

3.8. Okay, so — one bend changed everything

1. **Linear stacks collapse** (Equation 3.1); XOR proves four points can defeat any line.
2. **A neuron is a thresholded similarity score**: below the boundary silent, above it proportional. One neuron → one boundary; a layer → a polytope partition; two hinges → a bump.
3. **Bumps tile functions** (universal approximation) — but the theorem promises existence, not learnability, economy, or generalization.
4. **Depth bends boundaries**: regions grow exponentially with depth, polynomially with width. That is why the field is *deep* learning.
5. **The MLP is a class now** (`nn.Module`, foundation first), and it separates what no line can.

Exercises

1. **(Pencil.)** Generalize Equation 3.1: show by induction that any stack of L linear layers equals a single linear layer. Where exactly does inserting ReLU between layers break your proof?
2. **(Pencil.)** Using the bump construction, give explicit weights for a one-hidden-layer network that outputs approximately 1 on $[0, 1]$ and 0 outside $[-0.2, 1.2]$. How many hidden units did you need?
3. **(Code.)** Retrain the XOR network with `hidden_dim = 2` across ten different seeds. How often does it reach 100%? Inspect a failed run: what happened to its hidden units? (Hint: check how many are permanently silent.)
4. **(Code.)** In the polytope figure, count distinct activation patterns as you vary width at depth 1 versus depth at width 4 (keep total units comparable). Which grows the region count faster?
5. **(Code.)** Replace the moons MLP's ReLU with `nn.Identity()` and retrain. Explain the resulting boundary in one sentence using Equation 3.1.

4. Training: Loss and Stochastic Gradient Descent

We now have models worth training: linear regressors (Chapter 1), classifiers (Chapter 2), and multilayer perceptrons (Chapter 3). We have losses that tell each of them how bad they are, and we know the direction of improvement is the negative gradient. What remains is the question every practitioner faces daily: *how, exactly, do you run the descent* when the dataset has a million examples, the model has millions of knobs, and the loss landscape is no longer a friendly bowl?

This chapter is about the workhorse answer, stochastic gradient descent, and the small set of refinements (learning rates, batch sizes, momentum, Adam) that turn it from a theoretical idea into the algorithm that trains essentially everything in this book.

4.1. Where losses come from, one more time

Before optimizing a loss, remember why it is *that* loss. In Chapter 1 we saw that assuming Gaussian noise on a linear process makes maximum likelihood collapse into least squares; in Chapter 2 the same argument with a categorical distribution produced cross-entropy. This is the general pattern: **a loss function is a noise model in disguise.** Choose how you believe the data deviates from your model, and maximum likelihood hands you the loss. The loss is how we tell the model it is doing badly → choosing it well is not a detail, it is the supervision itself.

With the loss fixed, training is one line:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \; \mathcal{L}(\mathbf{w}) = \arg\min_{\mathbf{w}} \; \frac{1}{n} \sum_{i=1}^{n} \ell_i(\mathbf{w}), \tag{4.1}$$

where ℓ_i is the loss on example i. Everything that follows is about minimizing this average when n is enormous.

4.2. The computational bottleneck

Gradient descent, as we left it in Chapter 1, updates with the *full* gradient:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.2}$$

Look at the cost. One step touches every example; modern datasets have millions to billions of them, modern models have millions to billions of parameters, and training needs thousands of steps. One exact gradient per step is a luxury we cannot afford → we need to change strategy to scale up.

4.3. Stochastic gradient descent

The idea is almost cheeky: we do not need the exact gradient. A good approximation is enough. Instead of the expensive sum over all n examples, average over a small random **mini-batch** $\mathcal{B}$ — think 32 examples against a million:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.3}$$

Why does this work? Because the mini-batch gradient is **unbiased**: if each example is sampled uniformly, the expected contribution of example i to a batch equals its contribution to the full average, so

$$\mathbb{E}\big[\nabla \mathcal{L}_{\mathcal{B}}(\mathbf{w})\big] = \nabla \mathcal{L}(\mathbf{w}). \tag{4.4}$$

In expectation, the cheap step points exactly where the expensive step would have pointed. Each step is noisy; on average, the walk is right.

In practice the algorithm is:

1. Shuffle the training data.
2. Split it into mini-batches of size B.
3. For each batch: compute the average gradient, update the parameters.
4. One pass through all batches is an **epoch**; repeat for multiple epochs.

> **i** Honest fine print
>
> The unbiasedness proof assumes sampling *with* replacement. The algorithm above samples *without* replacement (shuffle, then walk the batches), for practical reasons: we want to visit every example and spread the contributions evenly. That mismatch breaks the simple proof, and making the theory rigorous for shuffled SGD is an active area of research. Keep the caveat in mind; keep using the shuffle.

4.4. The two zones of SGD

The noise gives SGD a characteristic two-phase behavior, and it is worth seeing rather than just hearing about. Far from the optimum, almost any batch tells you roughly the same thing → rapid, consistent progress. Close to the optimum, different batches start to disagree ("go left" — "no, go right"), and the iterate bounces around in what we will call the **region of confusion**:

```
import torch
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(6050)
x1 = torch.randn(80)
y1 = 2.5 * x1 - 1.0 + 0.4 * torch.randn(80)

def descend(batch: int | None, lr: float = 0.12, steps: int = 60):
    w, b, path = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        idx = torch.randperm(80)[:batch] if batch else slice(None)
        err = (w * x1[idx] + b) - y1[idx]
        path.append((float(w), float(b)))
        w = w - lr * 2 * (err * x1[idx]).mean()
        b = b - lr * 2 * err.mean()
    return np.array(path)

W, B = np.meshgrid(np.linspace(-1.5, 5.5, 90), np.linspace(-4, 3, 90))
L = ((y1.numpy()[None, None, :] -
      (W[..., None] * x1.numpy() + B[..., None])) ** 2).mean(-1)

plt.figure(figsize=(6.2, 4))
plt.contour(W, B, L, levels=25, cmap="Blues", alpha=0.7)
for batch, color, label in [(None, "#232D4B", "full batch"),
                            (4, "#E57200", "SGD, B = 4")]:
    p = descend(batch)
    plt.plot(p[:, 0], p[:, 1], "o-", ms=2.5, lw=1.2, color=color, label=label)
plt.plot(2.5, -1.0, "k*", ms=13, label="truth")
plt.xlabel("$w$"); plt.ylabel("$b$"); plt.legend(loc="upper right")
plt.tight_layout(); plt.show()
```

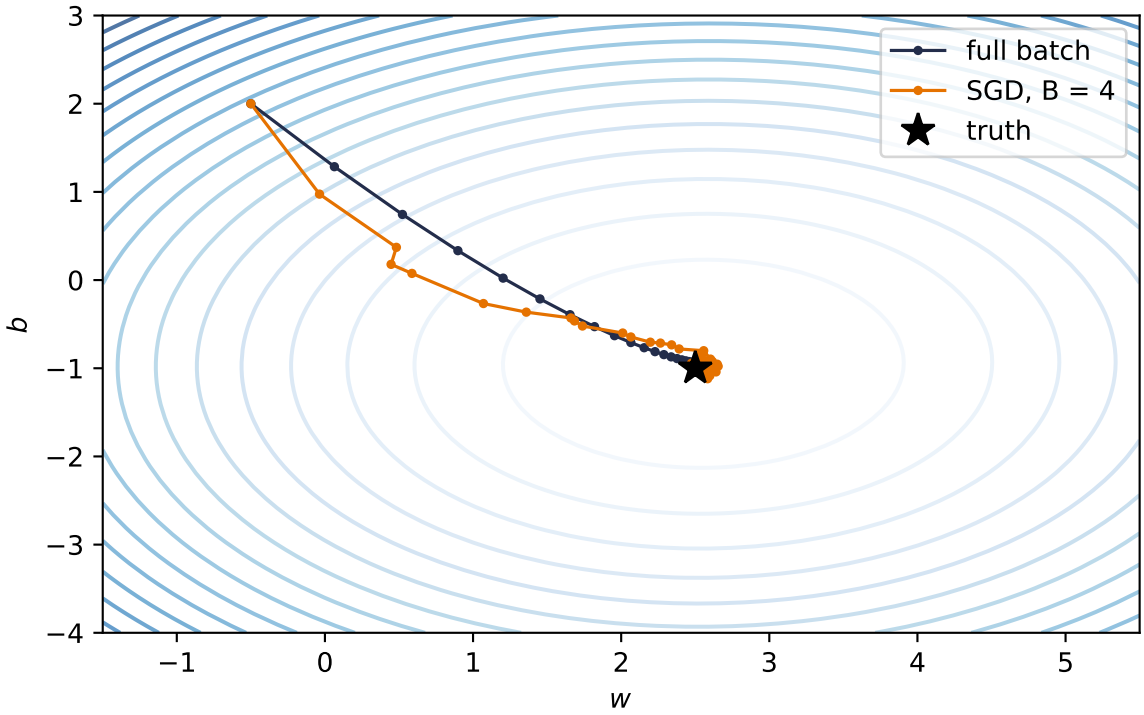

Figure 4.1.: Full-batch descent (blue) against mini-batch SGD (orange, $B = 4$) on the same landscape. Far out, the noisy steps agree with the true direction; near the optimum they scatter — the region of confusion.

Here is the surprise: the bouncing is not merely tolerable, it is often a **feature**. Loss landscapes of real networks are full of shallow traps, and a noisy step can hop out of a poor local minimum that would permanently capture exact gradient descent. The same jitter discourages the model from settling too perfectly into any one configuration of the training set; solutions found under noise tend to *generalize better*. We will give that observation its proper treatment in Chapter 6; for now, respect the noise. It is doing work for you.

4.5. The learning rate

One knob dominates all others. The learning rate α scales every step, and its failure modes are asymmetric: too small wastes your compute budget crawling; too large overshoots the valley and diverges.

```
def losses_for(lr: float, steps: int = 60):
    w, b, out = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        err = (w * x1 + b) - y1
        out.append(float((err ** 2).mean()))
        w = w - lr * 2 * (err * x1).mean()
```

```
        b = b - lr * 2 * err.mean()
    return out

plt.figure(figsize=(6.2, 3.4))
for lr, style, color in [(0.005, "-", "#5379AA"), (0.12, "-", "#E57200"),
                         (1.1, "--", "#722F37")]:
    plt.semilogy(losses_for(lr), style, color=color, label=f"$\\alpha = {lr}$")
plt.xlabel("step"); plt.ylabel("loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

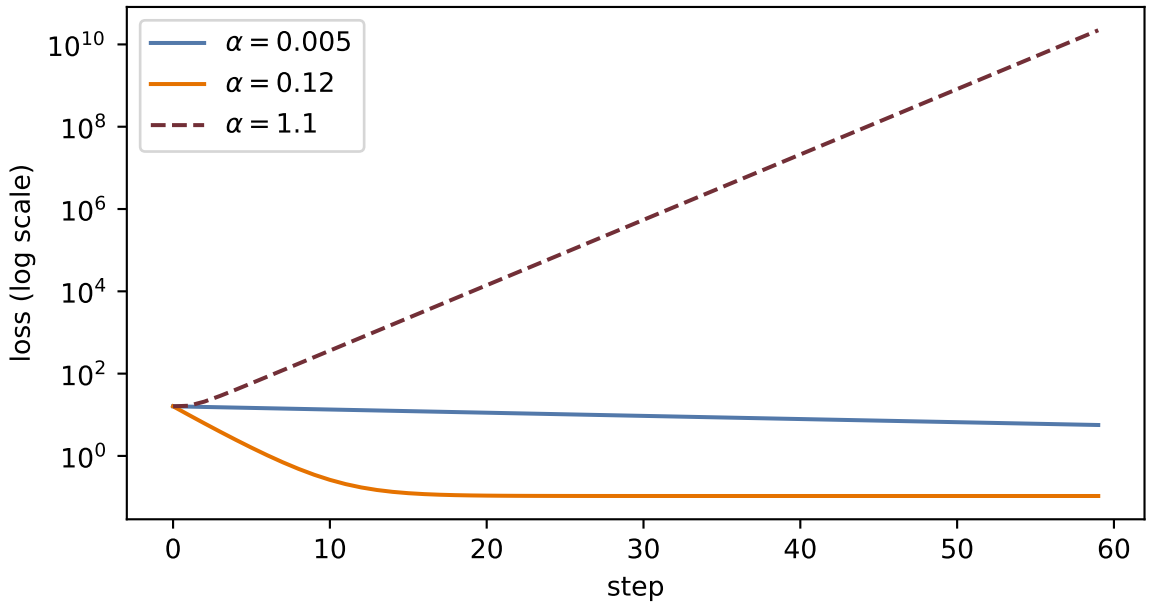

Figure 4.2.: Same problem, same steps, three learning rates. Too small crawls; too large overshoots back and forth and climbs; the middle one converges quickly.

> **Rule of thumb from the lectures**
>
> Start around $\alpha = 0.1$ and adjust based on what the training curves tell you: crawling → raise it; oscillating or exploding → lower it. Later in training it often pays to *decay* the learning rate so the fine-tuning steps get smaller — that is a **learning-rate schedule**. One exotic-sounding relative, *warmup* (starting tiny and ramping up), will matter enormously when we train Transformers in Chapter 14.

4.6. The batch size

The batch size B sets where you live on the noise–efficiency trade-off:

Batch size	Character	Trade-off
Small (1–32)	Noisy, exploratory	Better generalization; slow, underuses the GPU
Medium (32–256)	Balanced	The usual default; hardware-efficient
Large (256+)	Smooth, fast per epoch	Less exploration; can miss better minima and overfit

The noise level scales like $1/\sqrt{B}$: quadrupling the batch only halves the jitter. There is no free lunch here, only a dial — and the medium setting is the default for a reason.

4.7. Momentum: give the ball some mass

Vanilla SGD has exactly one control, α, and in narrow, curved valleys that is not enough: the iterate zigzags across the steep direction while inching along the shallow one. The fix is to give the update a memory. Keep a running **velocity** that accumulates gradients, and move along the velocity instead of the raw gradient:

$$\mathbf{v}^{(t)} = \beta\,\mathbf{v}^{(t-1)} + \nabla\mathcal{L}(\mathbf{w}^{(t)}), \qquad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\mathbf{v}^{(t)}, \tag{4.5}$$

with $\beta \in [0.9, 0.99]$. This is the ball rolling downhill: in directions where gradients keep agreeing, speed builds; in directions where they flip sign every step, they cancel. The zigzag damps itself, and small bumps in the landscape get rolled through rather than obeyed.

4.8. Adam: adaptive steps per knob

Momentum treats every parameter alike, but some knobs sit on steep cliffs while others sit on plains. **Adam** (adaptive moment estimation) combines three ideas from the lecture: momentum (accumulate past gradients), per-parameter adaptive learning rates (scale each knob's step by its own gradient history), and bias correction (account for both accumulators starting at zero). Formally, with elementwise operations:

$$\mathbf{m}^{(t)} = \beta_1\mathbf{m}^{(t-1)} + (1-\beta_1)\,\mathbf{g}^{(t)}, \quad \mathbf{s}^{(t)} = \beta_2\mathbf{s}^{(t-1)} + (1-\beta_2)\,\mathbf{g}^{(t)\,2}, \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{s}}^{(t)}} + \epsilon}, \tag{4.6}$$

where $\hat{\mathbf{m}}, \hat{\mathbf{s}}$ are the bias-corrected accumulators. When to use what, per the lecture: **SGD with momentum** is simple, reliable, and strong for large models; **Adam** is the good default that works out of the box.

4.9. The race, on a problem where we know the truth

Claims about optimizers deserve a test you can rerun. We build a least-squares problem with a deliberately *ill-conditioned* landscape — the second feature is ten times the scale of the first, so the loss surface is a long narrow valley — and race the three optimizers with the same budget:

```
torch.manual_seed(6050)
n = 256
Xr = torch.randn(n, 2) * torch.tensor([1.0, 10.0])   # ill-conditioned by design
w_true = torch.tensor([3.0, 0.3])
yr = Xr @ w_true + 0.1 * torch.randn(n)

def race(kind: str, steps: int = 120, B: int = 16):
    torch.manual_seed(1)                              # same batches for everyone
    w = torch.zeros(2, requires_grad=True)
    opt = {"SGD":      torch.optim.SGD([w], lr=3e-3),
           "momentum": torch.optim.SGD([w], lr=3e-3, momentum=0.9),
           "Adam":     torch.optim.Adam([w], lr=0.1)}[kind]
    hist = []
    for _ in range(steps):
        idx = torch.randint(0, n, (B,))
        opt.zero_grad()
        ((Xr[idx] @ w - yr[idx]) ** 2).mean().backward()
        opt.step()
        hist.append(((Xr @ w.detach() - yr) ** 2).mean().item())
    return hist

plt.figure(figsize=(6.2, 3.6))
for kind, color in [("SGD", "#5379AA"), ("momentum", "#232D4B"),
                    ("Adam", "#E57200")]:
    plt.semilogy(race(kind), color=color, label=kind)
plt.xlabel("step"); plt.ylabel("full-batch loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

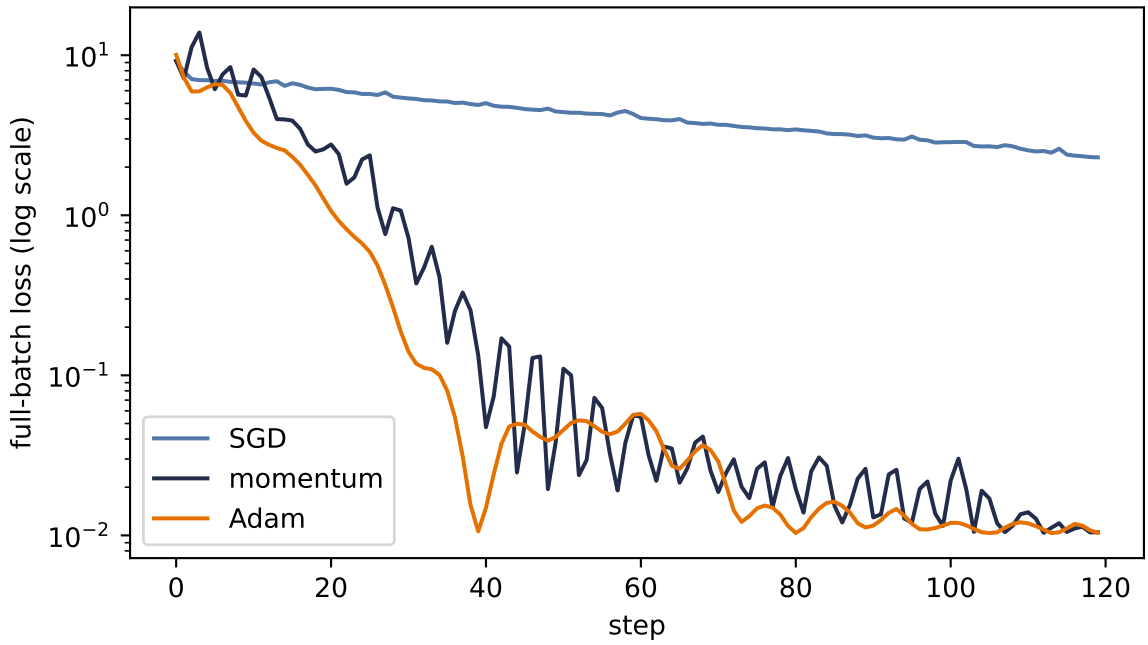

Figure 4.3.: The optimizer race on an ill-conditioned valley (feature scales 1 and 10). At the shared learning rate, plain SGD zigzags across the steep direction and crawls along the shallow one; momentum cancels the zigzag; Adam's per-parameter scaling neutralizes the conditioning almost entirely.

Read the result honestly. This is one problem, chosen to expose conditioning; it is not proof that Adam always wins (it does not — plain SGD with momentum, well tuned, still trains many of the best vision models). What the race does show is *mechanism*: when different directions of the landscape have wildly different steepness, per-direction memory (momentum) and per-parameter scaling (Adam) buy you orders of magnitude. It also whispers a practical lesson from the live session: much of this valley's narrowness came from unscaled features → *normalize your inputs* and you fight the optimizer less (Exercise 5).

4.10. Two regularizers that live inside training

Two ideas from Chapter 1 reappear here as training-loop citizens rather than loss-function algebra.

Weight decay. Ridge's L_2 penalty, seen from the optimizer's side, is one extra term in the update: shrink every weight slightly toward zero each step ($\mathbf{w} \leftarrow (1 - \alpha\lambda)\mathbf{w} - \alpha\nabla\mathcal{L}$). That is why every PyTorch optimizer takes a `weight_decay` argument — it is the same knob you met as ridge, now applied to any model, and you will see it in nearly every training recipe in this book.

Early stopping. Track the validation loss during training and stop when it turns upward, even though the training loss is still falling:

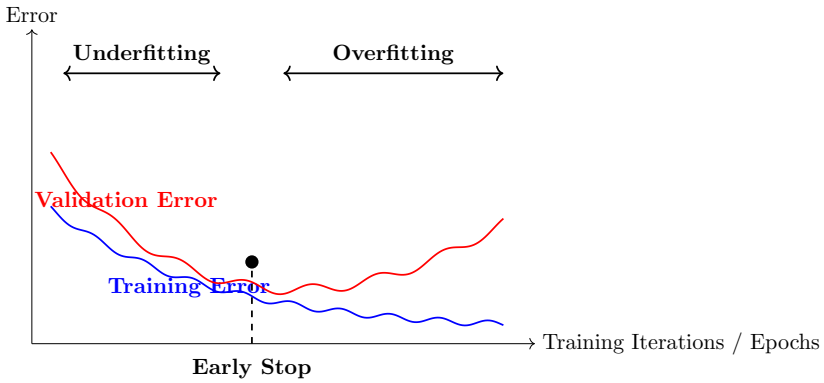

Figure 4.4.: Training error keeps falling; validation error turns. Stopping at the turn is regularization by clock — no penalty term required.

Both are cheap, both are everywhere, and both are previews: the full story of *why* constraining training improves generalization is the business of Chapter 6.

4.11. Okay, so — the training loop

Assemble the chapter into the loop you will run, in some form, for the rest of the book:

1. Start from (well-initialized) random parameters.
2. For each epoch: shuffle, split into mini-batches of size B.
3. For each batch: forward pass $\rightarrow$ loss (your noise model in disguise) $\rightarrow$ backward pass for the gradients $\rightarrow$ optimizer step (α, momentum or Adam, weight decay).
4. Watch the validation curve; schedule the learning rate; stop early when it turns.

One box in that loop is still magic: "backward pass for the gradients." Computing millions of partial derivatives at the cost of roughly one extra forward pass is the subject of Chapter 5.

Exercises

1. **(Pencil.)** Prove Equation 4.4 for sampling with replacement: if i is drawn uniformly from $\{1, \dots, n\}$, show $\mathbb{E}[\nabla \ell_i(\mathbf{w})] = \frac{1}{n} \sum_j \nabla \ell_j(\mathbf{w})$. Where exactly does the argument use uniformity?
2. **(Code.)** In the two-zones figure, sweep $B \in \{1, 4, 16, 80\}$ and plot the final 30 steps of each path. How does the radius of the region of confusion scale with B? Compare against the $1/\sqrt{B}$ rule.
3. **(Code.)** Add a learning-rate schedule to the race: halve α every 30 steps for plain SGD. How much of the gap to momentum does scheduling close? What does that tell you about what momentum is *really* fixing here?
4. **(Code.)** Sweep momentum's $\beta \in \{0, 0.5, 0.9, 0.99\}$ in the race. Explain the failure mode at the top end using the ball analogy.

5. **(Code.)** Standardize the race's features (divide each column of `Xr` by its standard deviation) and rerun all three optimizers. How much of Adam's advantage evaporates? State the practical moral in one sentence.

5. Backpropagation

Recall the update rule that ended Chapter 1: new parameters come from old parameters by stepping against the gradient, $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} \mathcal{L}$. For linear regression we could write that gradient down by hand. But a modern network has millions, sometimes billions, of knobs, tangled through layer after layer of composed functions → computing each partial derivative naively, one at a time, is hopeless.

Backpropagation is the algorithm that computes *all* of them, exactly, in one backward sweep whose cost is linear in the number of parameters. It is not a new kind of calculus. It is the chain rule, *organized*. By the end of this chapter you will have derived it, implemented it by hand, rebuilt it as a 60-line autograd engine, and then checked that `torch.autograd` agrees with you to machine precision.

5.1. One neuron, one chain

Start with the smallest possible case: one neuron on one training example. The previous activation $a^{(l-1)}$ is multiplied by a weight w, giving the pre-activation $z = w\,a^{(l-1)} + b$; the activation function produces $a = \sigma(z)$; and a feeds a loss L.

Now turn the knob w a little. How much does L change? Follow the chain: turning w changes z; the change in z changes a; the change in a changes L. The chain rule multiplies these local sensitivities:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}. \tag{5.1}$$

```
import matplotlib.pyplot as plt
import matplotlib.patches as mp

fig, ax = plt.subplots(figsize=(7.2, 2.4))
nodes = {r"$w$": 0, r"$z = w a^{(l-1)} + b$": 1, r"$a=\sigma(z)$": 2, r"$L$": 3}
for label, i in nodes.items():
    ax.add_patch(mp.FancyBboxPatch((i * 2.1, 0), 1.5, 0.7, boxstyle="round,pad=0.08",
                                   fc="#E8EEF7", ec="#232D4B", lw=1.4))
    ax.text(i * 2.1 + 0.75, 0.35, label, ha="center", va="center", fontsize=11)
fwd = dict(arrowstyle="-|>", color="#5379AA", lw=1.8)
bwd = dict(arrowstyle="-|>", color="#E57200", lw=1.8)
lbl = [r"$\frac{\partial z}{\partial w}$", r"$\frac{\partial a}{\partial z}$",
       r"$\frac{\partial L}{\partial a}$"]
```

```
for i in range(3):
    ax.annotate("", xy=(2.1 * (i + 1) - 0.1, 0.55), xytext=(2.1 * i + 1.6, 0.55),
                arrowprops=fwd)
    ax.annotate("", xy=(2.1 * i + 1.6, 0.12), xytext=(2.1 * (i + 1) - 0.1, 0.12),
                arrowprops=bwd)
    ax.text(2.1 * i + 1.85, -0.28, lbl[i], ha="center", fontsize=12, color="#E57200")
ax.set_xlim(-0.3, 8.3); ax.set_ylim(-0.7, 1.1); ax.axis("off")
plt.tight_layout(); plt.show()
```

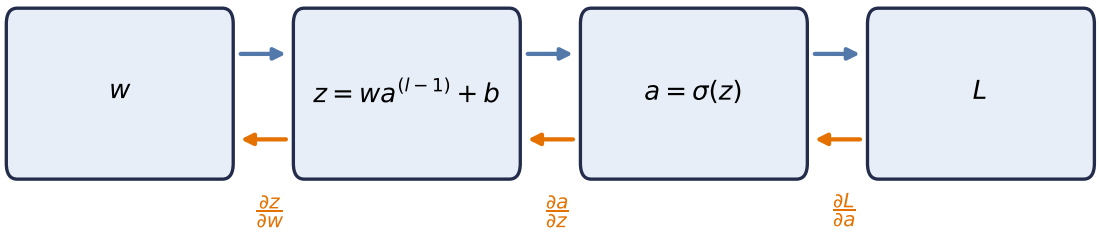

Figure 5.1.: Computation as a graph: the forward pass (top, blue) computes and caches values; the backward pass (bottom, orange) multiplies local derivatives along the same edges, in reverse.

Two things about this picture generalize to any network, however deep:

1. **The forward pass caches.** Computing L produces the intermediate values (z, a) along the way; we keep them, because the backward pass will need them.
2. **The backward pass reuses.** Each edge contributes one *local* derivative, and the derivative of L with respect to anything is the product of local derivatives along the path back to it. Nothing is recomputed → the whole sweep costs about as much as one forward pass.

5.2. The game of blame

For a full layer of neurons the bookkeeping threatens to become a tangled mess of sums. The cure is to pick the right intermediate quantity and track only that. At the output of the network the error is obvious: it is just the gap to the target. What is *not* obvious is how much of that error is the fault of a weight buried ten layers deep. Backpropagation sends this blame backward, from where it is obvious to where it is not.

The blame signal

Define, for each layer l, the **blame signal**

$$\delta^{(l)} := \frac{\partial L}{\partial \mathbf{z}^{(l)}},$$

the sensitivity of the loss to that layer's *pre-activations*. It answers: how much did each neuron's z contribute to the final loss?

Everything now unfolds in four short equations. With $\odot$ denoting elementwise (Hadamard) product:

1. Start where blame is obvious (output layer L):

$$\delta^{(L)} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \odot \sigma'(\mathbf{z}^{(L)}). \tag{5.2}$$

For squared error, $\partial L/\partial \mathbf{a}^{(L)}$ is literally the prediction error $\mathbf{a}^{(L)} - \mathbf{y}$: the error, scaled by how sensitive the activation was to its input.

2. Propagate blame one layer back:

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}\right) \odot \sigma'(\mathbf{z}^{(l)}). \tag{5.3}$$

Blame flows backward through the same weights the signal flowed forward through, hence the transpose; then the local gate $\sigma'(\mathbf{z}^{(l)})$ scales it. This recursion runs from the last layer to the first.

3. Convert blame into weight gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \, (\mathbf{a}^{(l-1)})^\top. \tag{5.4}$$

4. And into bias gradients:

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \tag{5.5}$$

Equation Equation 5.4 deserves a pause, because its shapes tell the story. With m neurons in layer l and n in layer $l-1$: $\delta^{(l)}$ is $(m \times 1)$, and $(\mathbf{a}^{(l-1)})^\top$ is $(1 \times n)$. Their **outer product** is an $m \times n$ matrix, exactly the shape of $\mathbf{W}^{(l)}$, and entry (i, j) is the blame of neuron i times the activation of neuron j that fed it. The update for a weight is *blame out times signal in*. No messy summations: the matrix form performs them all at once, and it maps directly onto the parallel matrix multiplies GPUs are built for. The same four equations govern a toy network and a billion-parameter model; only the matrix sizes change.

5.3. Build it, then check it against autograd

Let us implement the four equations on a two-layer network, with no autograd anywhere, and then ask `torch.autograd` to differentiate the same network. If our blame algebra is right, the two answers must agree to machine precision.

```
import torch

torch.manual_seed(6050)
```

```
<torch._C.Generator at 0x129a216b0>
```

```
n, d, h = 32, 5, 8                              # batch, input dim, hidden dim
X = torch.randn(n, d)
y = torch.randn(n)

W1, b1 = torch.randn(h, d) * 0.3, torch.zeros(h)
W2, b2 = torch.randn(1, h) * 0.3, torch.zeros(1)

# forward pass -- cache everything the backward pass will need
Z1 = X @ W1.T + b1                              # (n, h)
A1 = torch.sigmoid(Z1)                          # (n, h)
Z2 = A1 @ W2.T + b2                             # (n, 1)
L = ((Z2.squeeze() - y) ** 2).mean()

# backward pass -- the four equations (batch-averaged)
sig_prime = A1 * (1 - A1)                       # sigma'(z) = sigma(z)(1 - sigma(z))
delta2 = 2 * (Z2.squeeze() - y)[:, None] / n  # eq. 1 (identity output: no gate)
delta1 = (delta2 @ W2) * sig_prime              # eq. 2: blame through W^T, gated
grads = {                                       # eqs. 3 & 4: blame out x signal in
    "W2": delta2.T @ A1, "b2": delta2.sum(0),
    "W1": delta1.T @ X,  "b1": delta1.sum(0),
}

# same network, autograd's turn
params = {k: v.clone().requires_grad_(True) for k, v in
          {"W1": W1, "b1": b1, "W2": W2, "b2": b2}.items()}
A1_ = torch.sigmoid(X @ params["W1"].T + params["b1"])
L_ = (((A1_ @ params["W2"].T + params["b2"]).squeeze() - y) ** 2).mean()
L_.backward()

for k in grads:
    gap = (grads[k] - params[k].grad).abs().max()
    print(f"{k}: max |ours - autograd| = {gap:.2e}")
```

```
W2: max |ours - autograd| = 0.00e+00
b2: max |ours - autograd| = 0.00e+00
W1: max |ours - autograd| = 3.73e-09
b1: max |ours - autograd| = 1.86e-09
```

Zeros, to floating-point precision. The blame equations *are* what autograd computes; the framework simply builds the computation graph for us as we go and then walks it backward, caching and reusing exactly as we did.

5.4. A 60-line autograd engine

To remove the last trace of magic, let us build the graph ourselves. Each `Value` node remembers how it was made (its parents and the local derivative rule); calling `backward()` walks the graph in reverse topological order, accumulating blame into every node's `grad`.

```python
class Value:
    """A scalar that remembers its history -- a node in the computation graph."""

    def __init__(self, data: float, parents=(), backward_rule=lambda: None):
        self.data = data
        self.grad = 0.0
        self._parents = parents
        self._backward_rule = backward_rule

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other))
        def rule():                       # d(out)/d(self) = d(out)/d(other) = 1
            self.grad += out.grad
            other.grad += out.grad
        out._backward_rule = rule
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other))
        def rule():                       # product rule, one side at a time
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward_rule = rule
        return out

    def sigmoid(self):
        s = 1 / (1 + 2.718281828459045 ** (-self.data))
        out = Value(s, (self,))
```

```
        def rule():                         # sigma' = s (1 - s), the gate
            self.grad += s * (1 - s) * out.grad
        out._backward_rule = rule
        return out

    def backward(self):
        order, seen = [], set()
        def topo(v):                        # children before parents
            if v not in seen:
                seen.add(v)
                for p in v._parents:
                    topo(p)
                order.append(v)
        topo(self)
        self.grad = 1.0                     # dL/dL = 1: blame starts here
        for v in reversed(order):
            v._backward_rule()
```

```
w, x, b = Value(0.7), Value(2.0), Value(-0.5)
a = ((w * x) + b).sigmoid()             # one neuron, forward
loss = (a + (-0.3)) * (a + (-0.3))      # squared error vs target 0.3
loss.backward()

# analytic check: dL/dw = 2(a - y) * sigma'(z) * x
z = 0.7 * 2.0 - 0.5
s = 1 / (1 + 2.718281828459045 ** (-z))
print(f"micro-autograd: dL/dw = {w.grad:.6f}")
print(f"by hand:        dL/dw = {2 * (s - 0.3) * s * (1 - s) * 2.0:.6f}")
```

```
micro-autograd: dL/dw = 0.337801
by hand:        dL/dw = 0.337801
```

Sixty lines, and it is the real thing: PyTorch's autograd is this idea with tensors instead of scalars, a compiled graph walker, and thousands of derivative rules.

⚠ Autograd hygiene (the pitfalls that actually bite)

- **Gradients accumulate.** `backward()` *adds* into `.grad`; forget `optimizer.zero_grad()` and every step uses the sum of all past gradients.
- **Updates happen outside the graph.** Parameter updates go inside `torch.no_grad()`; otherwise the update itself is recorded and memory explodes.
- **`detach()` cuts the tape.** Use it when you want a value without its history (logging, plotting, building targets).

5.5. The gate, and the problem with deep chains

Look again at Equation 5.3. Every backward step multiplies by $\sigma'(\mathbf{z}^{(l)})$. That derivative acts as a **gate** on the blame: wide open, and the signal passes; nearly closed, and it dies.

Now examine the sigmoid's gate. From $\sigma'(z) = \sigma(z)(1-\sigma(z))$, its maximum value, at $z = 0$, is exactly 0.25; away from zero the neuron *saturates* and the gate approaches 0. The sigmoid gate is at best a quarter open, and often nearly closed.

```
import matplotlib.pyplot as plt

z = torch.linspace(-6, 6, 300)
sig = torch.sigmoid(z)
fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.9), sharey=True)
axes[0].plot(z, sig * (1 - sig), color="#232D4B")
axes[0].axhline(0.25, ls="--", color="#E57200", lw=1)
axes[0].text(2.1, 0.26, "ceiling: 0.25", color="#E57200", fontsize=9)
axes[0].set_title(r"sigmoid gate $\sigma'(z)$"); axes[0].set_xlabel("$z$")
axes[1].plot(z, (z > 0).float(), color="#232D4B")
axes[1].set_title(r"ReLU gate"); axes[1].set_xlabel("$z$")
plt.tight_layout(); plt.show()
```

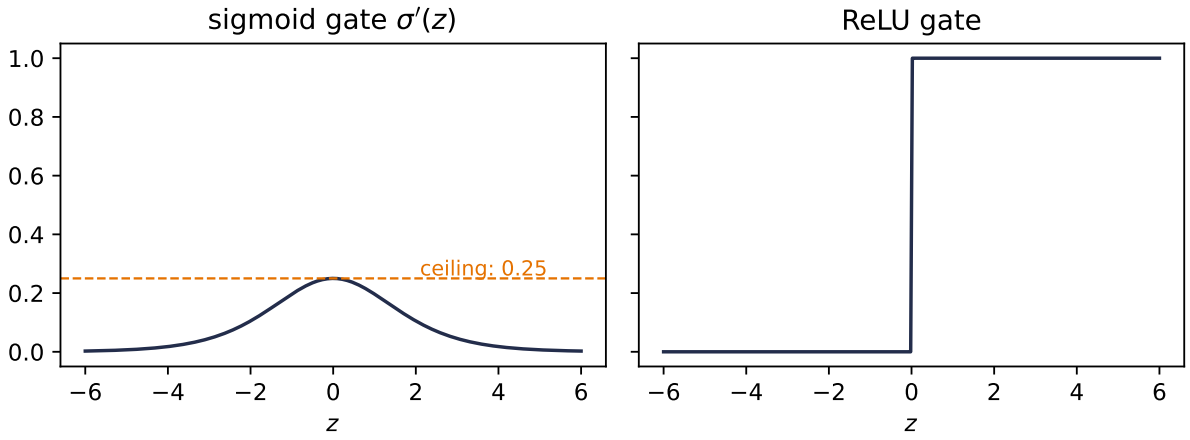

Figure 5.2.: Two gates. The sigmoid's derivative never exceeds 0.25 and vanishes under saturation; ReLU's is fully open (exactly 1) for every active neuron.

Here is the consequence, and it is the single most important failure mode in deep learning. The chain rule *multiplies* gates across layers → with sigmoid activations the backward signal shrinks by a factor of at most 0.25 per layer, so after k layers it is bounded by 0.25^k. Ten layers in, the ceiling is below one in a million. The blame reaching the early layers is too weak to update them: the **vanishing gradient** problem.

The fix is almost embarrassingly simple. $\mathrm{ReLU}(z) = \max(0, z)$ has derivative 1 for $z > 0$ and 0 otherwise (the kink at exactly zero never matters in floating point). The gate is either fully

open or fully closed. For every neuron that was active in the forward pass, blame passes through *unchanged*: a **gradient superhighway** through the network. This, more than anything, is why ReLU and its variants are the default in deep networks.

Do not take the argument on faith; measure it. We build the same 30-layer network twice (sigmoid vs. ReLU), push one batch through, and record how much gradient reaches each layer:

```
from torch import nn

def grad_by_layer(activation: type[nn.Module], depth: int = 30, width: int = 64):
    layers = []
    for _ in range(depth):
        layers += [nn.Linear(width, width), activation()]
    net = nn.Sequential(*layers, nn.Linear(width, 1))
    out = net(torch.randn(128, width)).mean()
    out.backward()
    return [lin.weight.grad.norm().item()
            for lin in net if isinstance(lin, nn.Linear)][:-1]

torch.manual_seed(6050)
plt.figure(figsize=(6.2, 3.4))
for act, color in [(nn.Sigmoid, "#232D4B"), (nn.ReLU, "#E57200")]:
    plt.semilogy(grad_by_layer(act), "o-", ms=3, color=color, label=act.__name__)
plt.xlabel("layer (0 = closest to input)")
plt.ylabel(r"$\|\partial L / \partial W^{(l)}\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

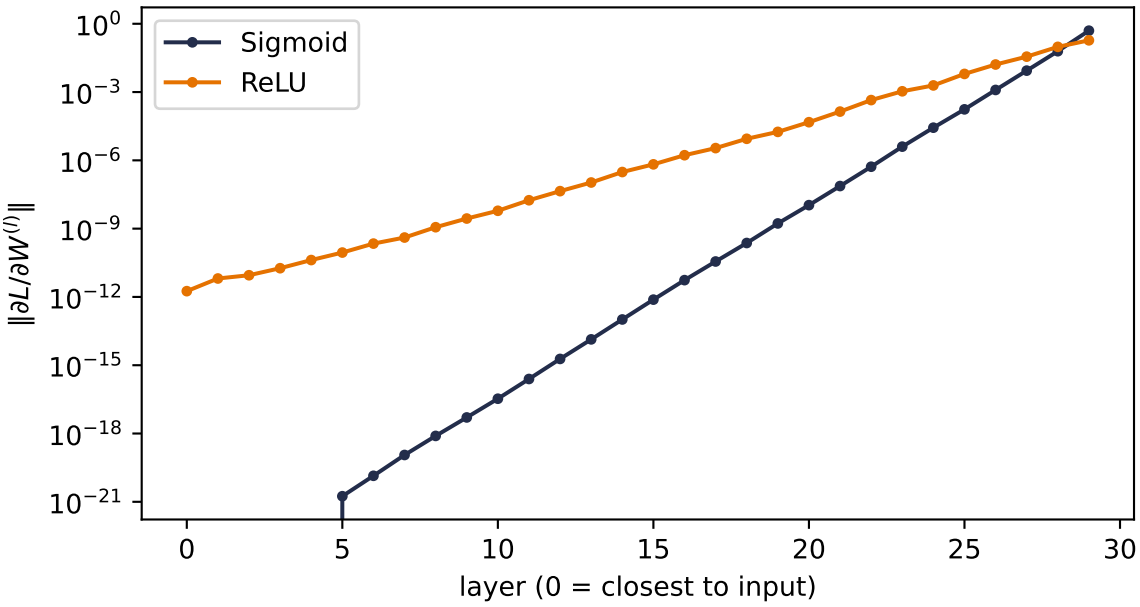

Figure 5.3.: Gradient reaching each layer of a 30-layer MLP (log scale). Depth attenuates both, but the sigmoid gates compound the decay so violently that below layer 5 the gradient falls beneath floating-point precision entirely, while ReLU's per-layer decay stays gentle enough for usable signal to reach the input.

Read the slopes: every step backward through a sigmoid layer multiplies the signal by its nearly-closed gate, and the compounding is so violent that by five layers from the input the gradient is numerically *zero* — those layers will never learn. ReLU is not magic (depth still attenuates the signal), but its open gates make the per-layer decay orders of magnitude gentler, and usable blame reaches all the way down. When we build deep MLPs for real in Chapter 3, this plot is the reason they train at all.

5.6. Initialization: setting the signal's energy

One more lever lives in this chapter, because it falls out of the same variance bookkeeping. Think of the network as a pipeline and the variance of the signal as its *energy*. If each layer multiplies the variance by more than one, activations explode after enough layers; by less than one, they vanish, and by Equation 5.3 the gradients follow. A good initialization keeps the energy roughly constant in both directions.

The analysis is one line. For $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ with independent, zero-mean inputs and weights, $\operatorname{Var}(z) = n_{\text{in}} \operatorname{Var}(w) \operatorname{Var}(x)$. Keeping $\operatorname{Var}(z) = \operatorname{Var}(x)$ demands

$$\operatorname{Var}(w) = \frac{1}{n_{\text{in}}} \quad \text{(forward)}, \qquad \operatorname{Var}(w) = \frac{1}{n_{\text{out}}} \quad \text{(backward)}.$$

Xavier initialization splits the difference for symmetric activations like tanh and sigmoid, $\mathrm{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$. **He initialization** accounts for ReLU discarding half its input distribution by doubling the forward recipe, $\mathrm{Var}(w) = 2/n_{\text{in}}$. PyTorch applies sensible defaults for you; the difference between a good and a careless choice is smooth training versus a dead or exploding network:

```
def signal_std(scale_fn, depth: int = 40, width: int = 256):
    x = torch.randn(512, width)
    stds = []
    for _ in range(depth):
        W = scale_fn(torch.randn(width, width), width)
        x = torch.relu(x @ W)
        stds.append(x.std().item())
    return stds

torch.manual_seed(6050)
schemes = {
    "naive  (std = 0.05)": lambda W, n: W * 0.05,
    "naive  (std = 0.12)": lambda W, n: W * 0.12,
    "He  (std = sqrt(2/n))": lambda W, n: W * (2 / n) ** 0.5,
}
plt.figure(figsize=(6.2, 3.4))
for (name, fn), color in zip(schemes.items(), ["#5379AA", "#722F37", "#E57200"]):
    plt.semilogy(signal_std(fn), color=color, label=name)
plt.xlabel("layer"); plt.ylabel("std of activations")
plt.legend(); plt.tight_layout(); plt.show()
```

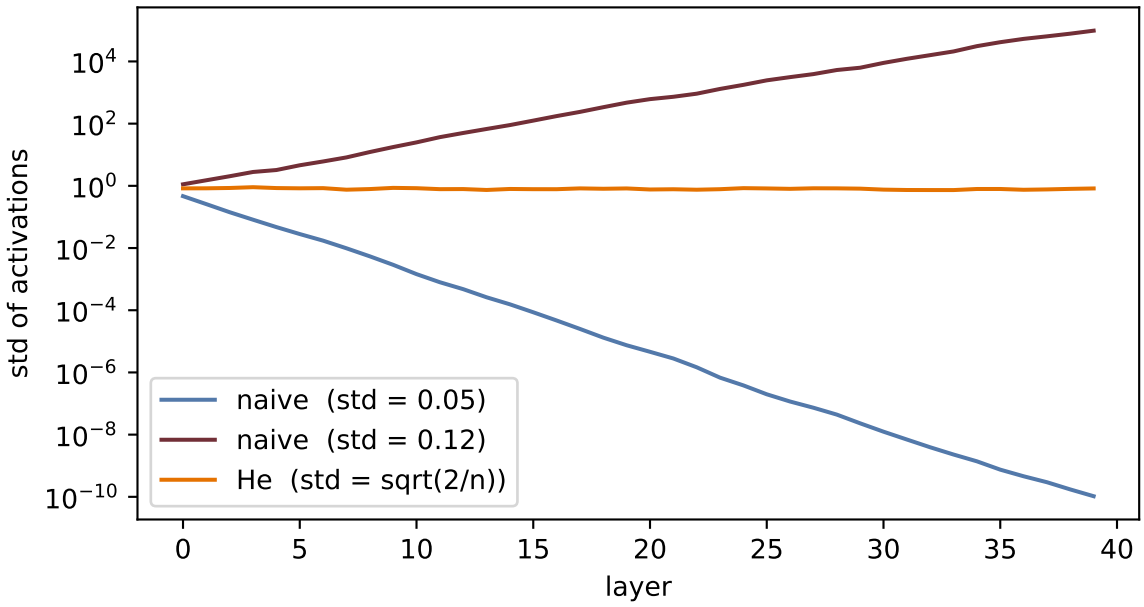

Figure 5.4.: Signal energy through 40 ReLU layers. Careless scales lose or amplify the signal exponentially; He initialization holds it steady.

Initialization gives a stable *start*. Keeping the signal stable *during* training, as the weights move, is the job of normalization layers; and for very deep networks even that is not enough, and gradients get an architectural bypass: skip connections that add a block's input straight to its output. Both belong to later chapters (normalization with training practice in Chapter 4 and Chapter 9; the residual idea in Chapter 9, and layer normalization where it becomes essential, inside the Transformer of Chapter 14). What you should carry from here is the invariant they all serve: *keep the signal, forward and backward, at constant energy.*

5.7. Okay, so — the chain rule, organized

1. **The graph.** Any network is a computation graph; the forward pass computes and caches, the backward pass multiplies local derivatives in reverse.
2. **The blame signal.** $\delta^{(l)} = \partial L/\partial \mathbf{z}^{(l)}$ turns the tangle into four equations: start at the output (Equation 5.2), propagate through $\mathbf{W}^\top$ and the gate (Equation 5.3), then read off weight and bias gradients (Equation 5.4, Equation 5.5). Cost: one extra forward pass, for *every* gradient at once.
3. **We verified it**: our hand-built equations match `torch.autograd` to 10^{-8}, and a 60-line `Value` class reproduces the machinery end to end.
4. **The gate rules the depth.** Multiplied sigmoid gates vanish like 0.25^k; ReLU's open gate is a gradient superhighway. Initialization sets the signal's energy so neither direction explodes or dies at the start.

Backpropagation is the engine under every remaining chapter. The next time a deep model fails to learn, your first questions now have names: *is the gradient reaching the early layers? what is*

gating it? what is its energy?

Exercises

1. **(Pencil.)** Derive the four blame equations for a network whose output layer uses the identity activation and squared-error loss (the network of our verification cell). Confirm that your $\delta^{(2)}$ matches the `delta2` line in the code.
2. **(Pencil.)** In Equation 5.3, why does the blame flow through $(\mathbf{W}^{(l+1)})^\top$ rather than $\mathbf{W}^{(l+1)}$? Argue from shapes: what are the dimensions of $\delta^{(l)}$, $\delta^{(l+1)}$, and $\mathbf{W}^{(l+1)}$?
3. **(Code.)** Extend the `Value` class with `tanh` and use it to train the one-neuron model from the check cell for 50 steps (write the update loop yourself). Verify the loss decreases.
4. **(Code.)** In the depth experiment, give the sigmoid network Xavier initialization (`nn.init.xavier_normal_` on each linear layer). How much of the vanishing does good initialization recover at depth 30? What does that tell you about why both fixes, initialization *and* ReLU, became standard?
5. **(Code.)** Comment out `optimizer.zero_grad()` in any training loop from Chapter 1 and describe what happens to the loss curve. Explain it with one sentence about `.grad` accumulation.

6. When It Fails: Generalization in Pictures, and Inductive Bias

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The signature chapter. We watch an MLP that aces MNIST fall apart when digits shift two pixels — in pictures. Capacity curves, sample complexity, the curse of dimensionality; and the cure: build what you know about the data into the architecture. That idea powers every remaining chapter.

Part II.

II · Vision: Learning the Filters

7. Filters and Convolution (Fixed Kernels)

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Before learning, engineering: Sobel edges, blurs, sharpeners. Convolution as structured, weight-shared linear maps — translation equivariance for free. Open with the Chapter 1 callback: a filter response is the dot-product similarity score of Chapter 1, slid across the image — the template is just still hand-made.

8. CNNs: Making the Filters Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Ask the question: what if the kernel entries were parameters? LeNet from scratch, padding/stride/pooling, receptive fields, and why a small CNN beats a big MLP on images.

9. Modern CNNs and Transfer Learning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

VGG's stacked 3×3s, 1×1 convolutions, Inception's parallel paths, BatchNorm, ResNet's identity highway — and the practical superpower: start from a pretrained backbone.

Part III.

III · Sequences

10. Sequences and Recurrence

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Order matters. Parameter sharing across time gives the RNN; products of Jacobians give vanishing gradients; gates (LSTM/GRU) give memory a highway.

11. Encoder–Decoder, Teacher Forcing, Beam Search

> **Warning**
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Compress a sentence into a vector, then unroll it in another language. Teacher forcing, scheduled sampling, beam search — and the fixed bottleneck that sets up everything in Part IV.

Part IV.

IV · Attention: Learning the Similarity

12. Kernel Regression: Attention Before It Was Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

A century-old idea: predict by averaging nearby evidence, weighted by similarity. Nadaraya–Watson in NumPy, bandwidths, and the observation that this is already attention — with a fixed similarity function. Open with the two Chapter 1 callbacks (prediction as a weighted combination of training targets via the projection view, and the dot product as similarity score) plus the Chapter 2 callback (softmax normalizes scores into weights).

13. Attention: Making the Kernel Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Replace the fixed kernel with learned projections: queries, keys, values. Additive vs scaled dot-product (and why GPUs prefer the latter), the $\sqrt{d}$ scaling, masking, multi-head. The seq2seq bottleneck falls.

14. Self-Attention and the Transformer

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Let a sequence attend to itself and recurrence becomes optional. Positional encodings as rotations, multi-head subspaces, LayerNorm and residuals, the FFN as associative memory — the full block, built and trained.

15. The BERT Moment: Pretraining as the New Regime

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Stop training from scratch. Masked language modeling, bidirectional context, fine-tuning — and why 2018 changed the default workflow of the field.

16. Vision Transformers and Scaling Laws

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Patches as tokens: the transformer eats images, needs more data (inductive bias strikes again — this time as a trade), and obeys power laws. Chinchilla's 20-tokens-per-parameter arithmetic.

Part V.

V · The Pretrained Era

17. Adapting Pretrained Models: Prompting, PEFT, Quantization

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

When the model is too big to fine-tune, get surgical: prompting and in-context learning, LoRA's low-rank updates, quantization and QLoRA.

18. Alignment and RL Fine-Tuning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

From next-token prediction to helpfulness: instruction tuning, preference learning, and RLHF in outline.

19. Generative Models: From PCA to Diffusion

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

One more make-it-learnable arc: PCA → autoencoder → VAE → diffusion (and GANs as the adversarial detour). Latents, ELBO, noise schedules.

A. Linear Algebra and the SVD

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The geometry the book leans on: matrices as transformations, eigenvectors, and the SVD as the master decomposition.

B. Tensors in Practice

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Shapes, broadcasting, indexing, and the layer-algebra habits that prevent 90% of PyTorch bugs.

C. Notation

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Symbols used throughout the book.

D. Floating Point and Machine Precision

> ⚠ Warning
>
> **Appendix stub.** Planned (author request, July 2026). See `docs/backlog.md` §1.

Every number in this book is a lie of finite precision. What a float actually is (sign, exponent, mantissa), machine epsilon, why $0.1 + 0.2 \neq 0.3$*, catastrophic cancellation, and the patterns deep learning uses to survive it: log-sum-exp (the trick inside every softmax), solving instead of inverting, and mixed-precision training (fp16/bf16) — the bridge to quantization in Chapter 17.*